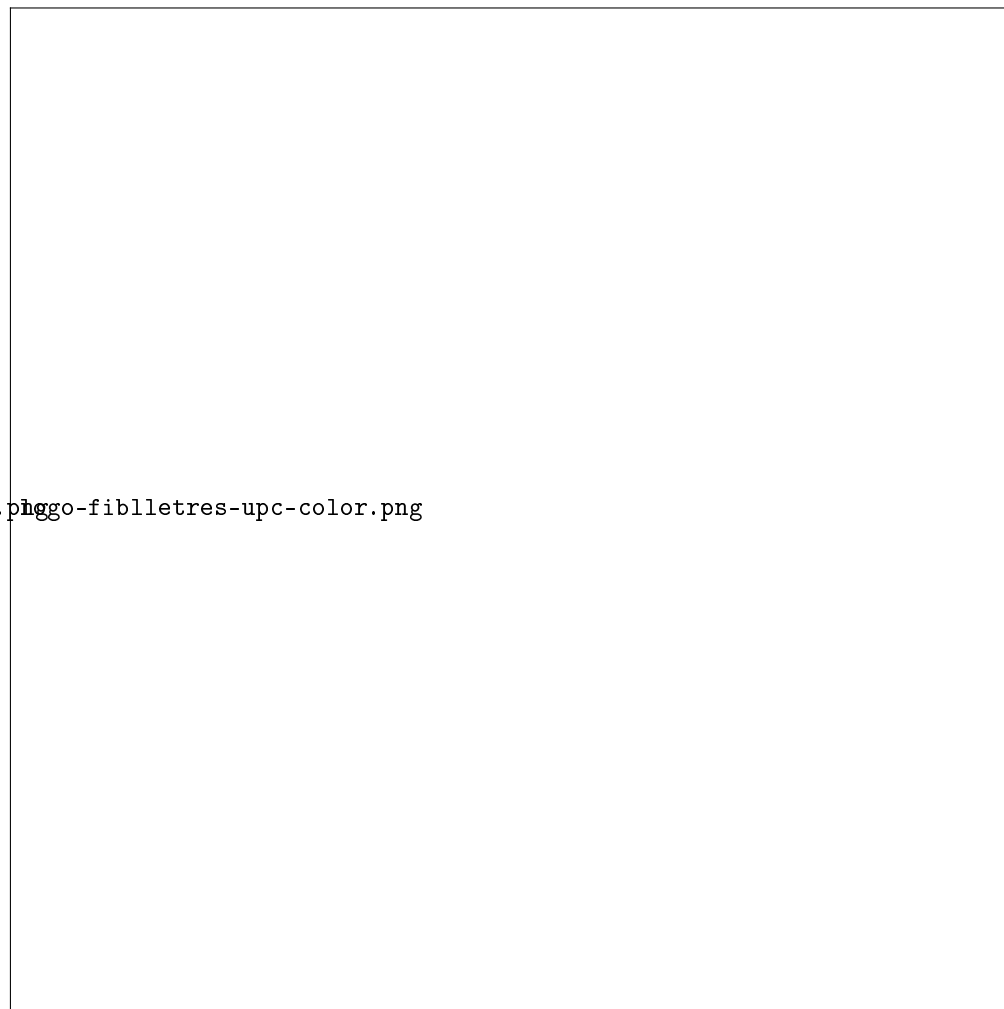


logo-upc.png



FACULTAD DE INFORMÁTICA DE BARCELONA (FIB)

UNIVERSIDAD POLITÉCNICA DE CATALUÑA (UPC)

ESPECIALIDAD EN TECNOLOGÍAS DE LA INFORMACIÓN

DISEÑO DE UN SISTEMA VISUAL PARA LA CREACIÓN Y DESPLIEGUE

DE ARQUITECTURAS MEDIANTE IAC

TRABAJO FINAL DE GRADO
INFORME DE SEGUIMIENTO DEL PROYECTO

Omar Antonio Cornejo Vargas

Director: Marc Catrisse i Pérez
Tutora GEP: Olga Pons Peregort
Titulación: Ingeniería Informática

21 de mayo de 2026

ÍNDICE

1. Introducción y contextualización

Este proyecto se desarrolla en el marco de la **Facultad de Informática de Barcelona (FIB)** de la **Universidad Politécnica de Cataluña (UPC)**, en el contexto de la especialización en Tecnologías de la Información. La propuesta se centra en aplicar los conocimientos adquiridos durante la titulación y aquellos aprendidos de forma autónoma, con el objetivo de abordar un problema real en el ámbito del **Cloud Computing** [1] y la **Infraestructura como Código (IaC)** [2].

1.1. Contextualización

El **Cloud Computing**, o computación en la nube permite a los usuarios acceder a los servicios de manera flexible, escalable y bajo demanda, sin la necesidad de mantener infraestructura local extensa. Esta modalidad simplifica la gestión de sistemas complejos, optimiza recursos y facilita la implementación rápida de soluciones en entornos dinámicos y cambiantes.

El Cloud Computing se ha convertido en un pilar fundamental para empresas de todos los tamaños, permitiendo lanzar productos y servicios con mayor rapidez y eficiencia, reducir costes de infraestructura y adaptarse a la demanda cambiante del mercado. Además, facilita la colaboración remota, la integración de tecnologías emergentes como inteligencia artificial, análisis de datos y aplicaciones distribuidas. Su adopción creciente transforma no solo la manera en que se gestionan los sistemas de información, sino también la forma de organizar los procesos empresariales y tecnológicos en un mundo cada vez más digitalizado.

1.2. Términos clave

A continuación se presentan los conceptos fundamentales que se utilizarán a lo largo de este proyecto. Comprender estos términos es esencial para entender tanto el contexto como la solución propuesta.

- **Cloud Computing (Computación en la nube):** El cloud computing es el acceso bajo demanda a recursos informáticos (servidores físicos o virtuales, almacenamiento de datos, capacidades de red, herramientas de desarrollo de aplicaciones, software, plataformas analíticas con IA y más) a través de Internet con precios de pago por uso.
- **Infraestructura como Código (IaC):** La infraestructura como código en DevOps es una herramienta poderosa para agilizar la configuración de la infraestructura, especialmente en entornos de nube. IaC automatiza el proceso de diseño y creación de la infraestructura.

- **Proveedor hiperescalar:** Los hyperscalers ofrecen servicios de cloud computing y de gestión de datos para las empresas que requieren una amplia infraestructura para el procesamiento y el almacenamiento de datos a gran escala. Si bien no hay un estándar universal para definir a los hyperscalers, los distribuidores de nube más importantes, como Amazon Web Services [3], Google Cloud [4], Microsoft Azure [5], IBM Cloud [6] y Alibaba Cloud [7], se ajustan a la descripción [8]
- **Escalabilidad:** La escalabilidad es la capacidad de un sistema, proceso o recurso para adaptarse y manejar un aumento en las demandas del usuario sin perder su rendimiento o funcionalidad [9].
- **Portabilidad:** La portabilidad es una característica del software, siendo una forma de reutilización que ayuda a evitar el “bloqueo” de ciertos entornos operativos, p.ej. proveedores de nube, sistemas operativos o vendedores [10].
- **Sostenibilidad:** La sostenibilidad informática va más allá de adoptar tecnologías ecológicas. Implica repensar cómo se gestionan los recursos digitales desde el hardware hasta la nube para optimizar el consumo, extender la vida útil de los equipos y fomentar la innovación sostenible en IT [11].
- **Open Source:** El software open source es un código diseñado de manera que sea accesible al público: todos pueden ver, modificar y distribuir el código de la forma que consideren conveniente [12].

Estos términos serán los más recurrentes en el desarrollo de la propuesta y son la base para entender los desafíos que enfrenta la industria y la relevancia de la solución planteada en este proyecto.

1.3. Problema

A pesar de sus ventajas, los equipos que trabajan en entornos Cloud enfrentan barreras importantes para diseñar y desplegar infraestructuras de manera eficiente. Las herramientas existentes suelen ser fragmentadas, presentan curvas de aprendizaje elevadas y limitaciones de interoperabilidad, lo que complica los procesos y reduce la productividad. Además, los distintos proveedores hiperescalares, como **Amazon Web Services (AWS)**, **Google Cloud Platform (GCP)** y **Microsoft Azure**, ofrecen servicios con características, nomenclaturas y modelos de facturación diferentes, lo que dificulta la estandarización de procesos y la portabilidad de aplicaciones entre plataformas.

La gestión de entornos en la nube requiere priorizar la escalabilidad y la sostenibilidad a largo plazo, garantizando que los servicios puedan evolucionar y adaptarse a nuevas demandas. Esto hace

necesario que los proyectos incorporen desde el inicio herramientas que proporcionen flexibilidad y adaptabilidad en la infraestructura, permitiendo un crecimiento orgánico y eficiente de los sistemas.

Ante este escenario, se hace evidente la necesidad de soluciones más accesibles, intuitivas y de código abierto, que combinen facilidad de uso con agilidad en la implementación, contribuyendo a reducir la complejidad y mejorar la eficiencia en el diseño y despliegue de infraestructuras en la nube.

1.4. Actores implicados

El producto está dirigido y utilizado por distintos actores dentro del ecosistema Cloud, y sus resultados impactan a varios niveles:

- **Proveedores de servicios Cloud:** Empresas como **Amazon Web Services (AWS)**, **Google Cloud Platform (GCP)** y **Microsoft Azure**. Son los encargados de ofrecer los recursos necesarios para desplegar y operar las infraestructuras. Se benefician indirectamente al facilitar el uso eficiente de sus servicios y fomentar la adopción de la nube.
- **Usuarios y organizaciones:** Usuarios finales, equipos de desarrollo, operaciones y administración de sistemas que utilizarán el producto para diseñar, implementar y mantener aplicaciones e infraestructuras. Son los usuarios principales y se benefician directamente de la automatización, estandarización y eficiencia que el sistema proporciona.
- **Comunidad y ecosistema Open Source:** Contribuye al desarrollo de herramientas, buenas prácticas y documentación, facilitando la colaboración y el acceso a soluciones abiertas. Se beneficia del intercambio de conocimiento y de la mejora continua de las soluciones que el producto integra o promueve.

2. Relevancia y Justificación

El diseño y despliegue de infraestructuras en la nube cuenta actualmente con diversas herramientas consolidadas en la industria. Entre ellas destacan soluciones de Infraestructura como Código (IaC) como Terraform [13], que permiten definir y gestionar recursos mediante archivos de configuración, así como plataformas de modelado visual como Brainboard [14]. Asimismo, los principales proveedores hiperescaladores, como Amazon Web Services (AWS), Google Cloud Platform (GCP) y Microsoft Azure, ofrecen herramientas propias que permiten visualizar y gestionar los recursos desplegados dentro de sus respectivos entornos.

A pesar de la existencia de estas soluciones, cada una presenta limitaciones que dificultan cubrir simultáneamente criterios de accesibilidad, independencia de proveedor y facilidad de diseño. Las herramientas basadas en código, como Terraform, proporcionan un alto grado de control y automatización, pero requieren conocimientos técnicos avanzados y familiaridad con lenguajes de configuración específicos, lo que puede suponer una barrera de entrada para perfiles junior, estudiantes o usuarios que se inician en el ámbito del Cloud Computing. Por otro lado, las plataformas que ofrecen modelado visual de infraestructuras suelen estar asociadas a modelos de licencia propietarios o a costes de uso, lo que limita su accesibilidad y dificulta su utilización en entornos educativos o de experimentación.

Asimismo, las herramientas proporcionadas por los propios proveedores cloud permiten visualizar y administrar infraestructuras ya desplegadas, pero no están diseñadas principalmente para facilitar el diseño conceptual de arquitecturas desde cero ni para trabajar desde una perspectiva independiente del proveedor. Este enfoque puede favorecer situaciones de dependencia con proveedores y reducir la portabilidad de las arquitecturas diseñadas entre distintas plataformas cloud.

Desde una perspectiva educativa y económica, este enfoque también aporta un valor añadido relevante. Una herramienta open source y orientada al aprendizaje facilita la experimentación con arquitecturas cloud sin necesidad de asumir costes asociados a licencias comerciales ni depender de plataformas propietarias. Esto favorece su utilización en contextos académicos, en procesos de formación y en entornos de prototipado, contribuyendo a reducir las barreras de acceso al aprendizaje del Cloud Computing.

En este contexto, la simple adaptación de una herramienta existente no resulta suficiente para cubrir de forma conjunta estos requisitos. El desarrollo de una nueva solución permite plantear una herramienta que combine las ventajas de la Infraestructura como Código con un entorno visual intuitivo y accesible, manteniendo además un enfoque abierto y neutral respecto a los distintos proveedores cloud y orientado tanto al aprendizaje como a la experimentación tecnológica.

2.1. Comparativa con soluciones existentes

Para situar la propuesta frente al estado del arte, la Tabla ?? compara las soluciones más representativas en cinco ejes que reflejan los requisitos del proyecto: la disponibilidad de un **modelado visual**, la **independencia respecto al proveedor cloud**, la condición de **open source**, la capacidad de **generar IaC editable** por el usuario, y la posibilidad de **ejecutar íntegramente en local** sin depender de una plataforma remota.

Solución	Visual	Indep. proveedor	Open source	IaC editable	Local
Terraform CLI	No	Sí	Sí	Sí	Sí
Pulumi	No	Sí	Sí	Sí	Sí
AWS CDK	No	No	Sí	Sí	Sí
Brainboard	Sí	Sí	No	Parcial	No
Consolas AWS / GCP / Azure	Parcial	No	No	No	No
Solución propuesta	Sí	Sí	Sí	Sí	Sí

Cuadro 1: Comparativa de soluciones existentes frente a la propuesta
Fuente: elaboración propia.

La tabla evidencia que cada solución cubre un subconjunto de los criterios, pero ninguna los reúne simultáneamente. Las herramientas basadas en código (Terraform, Pulumi [15], AWS CDK [16]) son potentes y locales, pero carecen de modelado visual y, en el caso de CDK, están ligadas a un proveedor concreto. Brainboard aporta el componente visual y la independencia de proveedor, pero es propietaria y depende de su plataforma remota. Las consolas oficiales de los hiperescaladores son visuales y gráficas, pero atan al usuario al proveedor y no generan IaC editable que pueda versionarse. La solución propuesta se sitúa precisamente en el hueco que estas herramientas dejan sin cubrir: un editor visual, open source, neutral respecto al proveedor, que produce IaC editable y se ejecuta en local.

3. Alcance del proyecto

3.1. Objetivo general

Diseñar y desarrollar un sistema visual que permita modelar arquitecturas Cloud de forma intuitiva y generar automáticamente la correspondiente definición de Infraestructura como Código (IaC), promoviendo la accesibilidad, la interoperabilidad entre proveedores y el enfoque open source.

No obstante, debido a las limitaciones temporales y al amplio alcance potencial del proyecto, se plantea como un producto mínimo viable (MVP) [17]. En consecuencia, se implementará un conjunto reducido y bien delimitado de funcionalidades esenciales, priorizando aquellas que demuestren la viabilidad del núcleo del sistema: la representación visual de recursos y la generación automática de IaC.

3.2. Subobjetivos

Una vez definido el objetivo general del proyecto, se establecen los siguientes subobjetivos específicos, orientados a concretar y estructurar el desarrollo de forma progresiva:

- Diseñar una interfaz gráfica que permita crear arquitecturas Cloud mediante componentes visuales reutilizables e intuitivos, con soporte para la conexión entre recursos.
- Implementar un mecanismo de traducción automática del diseño visual a código de IaC estructurado y coherente, garantizando la correspondencia semántica entre ambas representaciones.
- Definir una arquitectura de software modular y extensible que facilite la incorporación futura de nuevos proveedores Cloud y tipos de recursos.
- Validar el sistema mediante casos de uso representativos que demuestren su viabilidad técnica y su utilidad práctica en escenarios reales de despliegue.

3.3. Requisitos funcionales y no funcionales

Desde el punto de vista funcional, el sistema debe permitir la creación, edición y eliminación de recursos Cloud desde la interfaz gráfica, representar visualmente las relaciones y dependencias entre dichos recursos, y generar automáticamente archivos de configuración compatibles con herramientas

de IaC. Asimismo, debe ofrecer retroalimentación inmediata al usuario sobre la validez del diseño construido.

En cuanto a los requisitos no funcionales, la **usabilidad** y la **accesibilidad** son aspectos centrales del diseño: la interfaz debe ser clara e intuitiva, orientada tanto a perfiles técnicos como formativos, e inclusiva para distintos tipos de usuario. La **portabilidad** exige un diseño agnóstico respecto al proveedor Cloud, evitando dependencias que limiten su adopción. Por su parte, la **escalabilidad** y la **mantenibilidad** condicionan las decisiones arquitectónicas: el sistema debe estar preparado para incorporar nuevos servicios y su código ha de ser estructurado y documentado. Todo ello bajo un modelo de **licenciamiento open source** que garantice la libre distribución y reutilización del proyecto.

3.4. Obstáculos y riesgos

El desarrollo del proyecto presenta varios riesgos que conviene identificar y acotar desde el inicio. El principal reto técnico reside en la traducción precisa entre el modelo visual y el código IaC: las diferencias estructurales entre proveedores Cloud dificultan la definición de una abstracción común y coherente, y cualquier inconsistencia en esta capa puede comprometer la corrección del código generado. En segundo lugar, la complejidad inherente a la representación de dependencias entre recursos puede derivar en un modelo de datos excesivamente complejo si no se delimita con rigor desde el diseño. Finalmente, las limitaciones de tiempo y recursos propias de un Trabajo Final de Grado condicionan el alcance de lo que es posible abordar.

Para mitigar estos riesgos, se adoptarán las siguientes estrategias: definir una capa de abstracción común antes de implementar generadores específicos por proveedor; acotar el MVP a un subconjunto representativo pero manejable de recursos y relaciones; y establecer hitos parciales de revisión que permitan ajustar el alcance de forma controlada a lo largo del desarrollo.

4. Metodología de trabajo

Para el desarrollo de este proyecto se ha decidido combinar metodologías ágiles adaptadas al contexto individual de un TFG. Se aplicará Scrum [18] para organizar sprints pactados con el director del proyecto, y Kanban para gestionar internamente las tareas de desarrollo de manera visual y flexible. Esta combinación responde a las particularidades del proyecto: Scrum aporta la cadencia y los puntos de control necesarios para un desarrollo supervisado, mientras que Kanban permite gestionar la carga de trabajo diaria sin la rigidez de un equipo multidisciplinar, dado que el proyecto lo desarrolla una sola persona.

4.1. Estructura del trabajo

El proyecto se organiza en iteraciones o sprints de aproximadamente dos semanas, cada uno con objetivos claros y entregables definidos. Al inicio de cada sprint se planifican las tareas junto con el director, estableciendo metas alcanzables y priorizando el trabajo más relevante según el estado del desarrollo. Al finalizar cada iteración, se presenta el progreso obtenido y se recoge la retroalimentación del director, lo que permite ajustar el alcance y reorientar el siguiente sprint si fuera necesario.

Esta adaptación de Scrum difiere del marco original en varios aspectos deliberados: aunque el proyecto lo desarrolla el autor solo, se distinguen tres perfiles funcionales que se asume en distintos momentos del desarrollo: el de **jefe de proyecto** (planificación y seguimiento), el de **ingeniero de software** (diseño e implementación) y el de **redactor técnico** (documentación y defensa). Esta distinción, en lugar de los roles formales de Scrum (Product Owner, Scrum Master, desarrollador), permite asignar responsabilidades de forma explícita y trazable a cada tarea. Las ceremonias se simplifican a una reunión de revisión por sprint, y el backlog lo gestiona directamente el propio autor. Estas adaptaciones son coherentes con las limitaciones de un TFG individual y permiten mantener los beneficios del marco ágil — iteratividad, visibilidad del progreso y capacidad de replanificación — sin incurrir en una sobrecarga de proceso innecesaria.

4.2. Herramientas de seguimiento y validación

Las siguientes herramientas se utilizan no solo para organizar el trabajo, sino específicamente para validar que se alcanzan los objetivos definidos en el alcance del proyecto:

- **Control de versiones (GitHub) [19]:** permite mantener trazabilidad de todos los cambios

del código y verificar que cada subobjetivo técnico se traduce en commits y ramas concretas. El historial de commits actúa como evidencia objetiva del progreso real del proyecto.

- **Tablero Kanban (Trello) [20]:** organiza las tareas en columnas de estado (pendiente, en progreso, completada), permitiendo visualizar en todo momento si el avance del sprint es coherente con los objetivos planificados y detectar bloqueos de forma temprana.
- **Cronograma y diagramas de Gantt:** establecen los hitos temporales del proyecto y permiten comparar periódicamente el progreso real frente al planificado, facilitando la detección de desviaciones y la toma de decisiones sobre el alcance [21].

La combinación de estas herramientas crea un sistema de seguimiento en dos niveles: el nivel de tarea (Trello) y el nivel de hito (Gantt y revisiones de sprint), garantizando que tanto el día a día del desarrollo como los objetivos globales del proyecto estén siempre bajo control.

5. Planificación temporal

El proyecto se desarrolla entre el 9 de febrero y el 15 de junio de 2026, lo que representa aproximadamente 18 semanas de trabajo efectivo. La fase de gestión de proyecto ocupa las primeras semanas del calendario. Una vez concluida, las horas restantes se destinan al desarrollo técnico del prototipo, las iteraciones de revisión con el director y la redacción de la memoria y preparación de la defensa oral.

La planificación se organiza en sprints de dos semanas siguiendo la metodología ágil. La documentación se genera de forma continua en cada sprint, y las reuniones con el director se celebran cada dos semanas durante toda la duración del TFG. A continuación, se detallan los recursos necesarios para llevar a cabo correctamente el proyecto.

5.1. Recursos del proyecto

5.1.1. Recursos humanos

Se identifican tres roles que se asumen en distintos momentos del proyecto:

- **Jefe de proyecto (JP).** Responsable de la planificación, seguimiento del cronograma y gestión de riesgos.
- **Ingeniero de software (IS).** Encargado del diseño arquitectónico, la implementación del prototipo y la investigación tecnológica.
- **Redactor técnico (RT).** Responsable de la redacción de la memoria, la documentación técnica y la preparación de la defensa oral.

Adicionalmente, el director Marc Catrisse i Pérez actúa como colaborador externo, participando en las reuniones de seguimiento y validando las decisiones de diseño y el alcance del prototipo.

5.1.2. Recursos materiales

Hardware

- Torre PC (AMD Ryzen 7 5800X, NVIDIA RTX 3060 12 GB, 32 GB RAM DDR4, 1 TB NVMe SSD)

- Monitor HP Omen
- Monitor Dell
- Teclado y ratón

Software

- **VS Code** — entorno de desarrollo integrado [22].
- **Rust + Tauri** — lenguaje e infraestructura del backend [23] [24].
- **Vite + React, TypeScript** — stack del frontend [25] [26] [27].
- **GitHub** — control de versiones y gestión del repositorio
- **Trello** — tablero Kanban para el seguimiento de tareas
- **L^AT_EX** — redacción de la memoria del proyecto [28].
- **Google Meet y correo electrónico** — comunicación con el director [29].

5.1.3. Recursos de infraestructura

- **Espacio de trabajo:** domicilio del autor, utilizado como oficina durante toda la duración del proyecto.
- **Electricidad:** consumo eléctrico de los dispositivos del puesto de trabajo durante las 540 horas del proyecto.
- **Material de oficina:** suministros generales de papelería y consumibles necesarios durante el desarrollo.

5.2. Descripción de las tareas

Por cada tarea se indica descripción, estimación en horas, dependencias, recursos materiales y rol humano responsable:

R1 Ordenador personal con Linux e Internet

R2 L^AT_EX

R3 Google Meet y correo electrónico

R4 Trello

R5 GitHub

JP Jefe de proyecto

IS Ingeniero de software

RT Redactor técnico

5.2.1. GEP – Gestión del Proyecto

GEP (18 febrero 2026 → 18 marzo 2026), 90 h. Elaboración del documento de gestión del proyecto, definiendo alcance y objetivos, planificando tareas y tiempos, estimando costes y sostenibilidad, e integrando el feedback de los tutores.

Dep.: – / Rec.: R1, R2, R3 / Rol: JP, RT

5.2.2. DT – Desarrollo Técnico

Dedicación estimada: 450 h. Comprende la investigación, el aprendizaje tecnológico y la implementación iterativa del prototipo.

- **Sprint 1 – Investigación y definición inicial** (18 marzo 2026 → 1 abril 2026) 60 h
 - **DT.1 – Investigación y estado del arte** (15 h | Dep.: – | Rec.: R1 | Rol: IS)
Análisis de herramientas IaC (Terraform, Pulumi, CDK) y modelado visual para detectar limitaciones y oportunidades.
 - **DT.2 – Definición de requisitos y user stories** (25 h | Dep.: DT.1 | Rec.: R1, R3 | Rol: JP, IS)
Definición y refinamiento de requisitos e historias de usuario con feedback del director.
 - **DT.3 – Selección tecnológica** (10 h | Dep.: DT.1 | Rec.: R1 | Rol: IS)
Evaluación y justificación del stack tecnológico y lenguaje de programación.
 - **DT.4 – Seguimiento y documentación del sprint** (10 h | Dep.: – | Rec.: R1, R2, R3, R4 | Rol: JP, RT)
Reuniones periódicas con el director, junto con el registro de actividades, resultados y trazabilidad del sprint.
- **Sprint 2 – Aprendizaje tecnológico y diseño arquitectónico** (1 abril 2026 → 15 abril 2026) 65 h

- **DT.5 – Aprendizaje de Rust, Tauri y React** (30 h | Dep.: DT.3 | Rec.: R1, R5 | Rol: IS)
Estudio de Rust, Tauri y React, con ejercicios y prototipos.
- **DT.6 – Diseño de la arquitectura del sistema** (15 h | Dep.: DT.5 | Rec.: R1, R5 | Rol: IS)
Definición de arquitectura modular (frontend React, lógica de grafos y generación IaC en Rust).
- **DT.7 – Configuración del entorno** (10 h | Dep.: DT.3 | Rec.: R1, R5 | Rol: IS)
Preparación del entorno y estructura base del proyecto.
- **DT.8 – Seguimiento y documentación del sprint** (10 h | Dep.: – | Rec.: R1, R2, R3, R4 | Rol: JP, RT)
Reuniones periódicas con el director, junto con el registro de actividades, resultados y trazabilidad del sprint.
- **Sprint 3 – Primeras pruebas técnicas y mockups** (15 abril 2026 → 29 abril 2026) 70 h
 - **DT.9 – Canvas básico** (45 h | Dep.: DT.6, DT.7 | Rec.: R1, R4, R5 | Rol: IS)
Implementación inicial del editor visual con renderizado y drag-and-drop básico.
 - **DT.10 – Pruebas de traducción a IaC** (15 h | Dep.: DT.9 | Rec.: R1, R4, R5 | Rol: IS)
Primeras pruebas de generación de código HCL [30] desde elementos visuales.
 - **DT.11 – Seguimiento y documentación del sprint** (10 h | Dep.: – | Rec.: R1, R2, R3, R4 | Rol: JP, RT)
Reuniones periódicas con el director, junto con el registro de actividades, resultados y trazabilidad del sprint.
- **Sprint 4 – Continuación de la implementación** (29 abril 2026 → 13 mayo 2026) 90 h
 - **DT.12 – Editor visual completo** (30 h | Dep.: DT.9 | Rec.: R1, R4, R5 | Rol: IS)
Propiedades editables, zoom/pan, selección múltiple y persistencia del grafo.
 - **DT.13 – Biblioteca de recursos Cloud** (25 h | Dep.: DT.12 | Rec.: R1, R4, R5 | Rol: IS)
Paleta de componentes Cloud reutilizables, con cobertura inicial de AWS.
 - **DT.14 – Gestión de conexiones** (25 h | Dep.: DT.12 | Rec.: R1, R4, R5 | Rol: IS)
Aristas del grafo y validación de conexiones entre nodos, base para el código IaC.
 - **DT.15 – Seguimiento y documentación del sprint** (10 h | Dep.: – | Rec.: R1, R2, R3, R4 | Rol: JP, RT)
Reuniones periódicas con el director, junto con el registro de actividades, resultados y trazabilidad del sprint.
- **Sprint 5 – Mejoras a la implementación** (13 mayo 2026 → 27 mayo 2026) 85 h

- **DT.16 – Generador Terraform y UI** (25 h | Dep.: DT.10, DT.14 | Rec.: R1, R4, R5 | Rol: IS)
Motor en Rust para traducir el grafo a HCL y previsualización en UI.
- **DT.17 – Integración y empaquetado** (25 h | Dep.: DT.16 | Rec.: R1, R4, R5 | Rol: IS)
Persistencia de proyectos, exportación IaC y empaquetado en Linux.
- **DT.18 – Validación y seguridad** (25 h | Dep.: DT.16 | Rec.: R1, R4, R5 | Rol: IS)
Pruebas funcionales y análisis con Trivy.
- **DT.19 – Seguimiento y documentación del sprint** (10 h | Dep.: – | Rec.: R1, R2, R3, R4 | Rol: JP, RT)
Reuniones periódicas con el director, junto con el registro de actividades, resultados y trazabilidad del sprint.
- **Sprint 6 – Memoria y preparación de la defensa** (27 mayo 2026 → 15 junio 2026) 80 h
 - **DT.20 – Memoria** (55 h | Dep.: DT.19 | Rec.: R1, R2, R3, R4 | Rol: RT)
Redacción y revisión final del TFG.
 - **DT.21 – Preparación de la defensa oral** (15 h | Dep.: DT.20 | Rec.: R1, R4 | Rol: JP, RT)
Elaboración de diapositivas, preparación de la demostración del prototipo y ensayo de la exposición final.
 - **DT.22 – Seguimiento del sprint** (10 h | Dep.: – | Rec.: R1, R2, R3, R4 | Rol: JP, RT)
Reuniones periódicas con el director.

5.3. Tabla resumen de tareas

La siguiente tabla recoge de forma consolidada todas las tareas del proyecto, indicando para cada una su identificador, duración estimada, dependencias, recursos materiales y rol humano responsable.

ID	Tarea	Horas	Dep.	Recursos	Rol
GEP	Gestión del Proyecto	90	–	R1, R2, R3	JP, RT
DT.1	Investigación y estado del arte	15	–	R1	IS
DT.2	Definición de requisitos	25	DT.1	R1, R3	JP, IS
DT.3	Selección tecnológica	10	DT.1	R1	IS
DT.4	Seguimiento sprint 1	10	–	R1, R2, R3, R4	JP, RT
DT.5	Aprendizaje Rust, Tauri y React	30	DT.3	R1, R5	IS
DT.6	Diseño de la arquitectura	15	DT.5	R1, R5	IS
DT.7	Configuración del entorno	10	DT.3	R1, R5	IS
DT.8	Seguimiento sprint 2	10	–	R1, R2, R3, R4	JP, RT
DT.9	Canvas básico	45	DT.6, DT.7	R1, R4, R5	IS
DT.10	Pruebas de traducción a IaC	15	DT.9	R1, R4, R5	IS
DT.11	Seguimiento sprint 3	10	–	R1, R2, R3, R4	JP, RT
DT.12	Editor visual completo	30	DT.9	R1, R4, R5	IS
DT.13	Biblioteca de recursos Cloud	25	DT.12	R1, R4, R5	IS
DT.14	Gestión de conexiones	25	DT.12	R1, R4, R5	IS
DT.15	Seguimiento sprint 4	10	–	R1, R2, R3, R4	JP, RT
DT.16	Generador Terraform y UI	25	DT.10, DT.14	R1, R4, R5	IS
DT.17	Integración y empaquetado	25	DT.16	R1, R4, R5	IS
DT.18	Validación y seguridad	25	DT.16	R1, R4, R5	IS
DT.19	Seguimiento sprint 5	10	–	R1, R2, R3, R4	JP, RT
DT.20	Memoria	55	DT.19	R1, R2, R3, R4	RT
DT.21	Preparación de la defensa	15	DT.20	R1, R4	JP, RT
DT.22	Seguimiento sprint 6	10	–	R1, R2, R3, R4	JP, RT
Total		540			

Cuadro 2: Tabla resumen de tareas del proyecto
Fuente: elaboración propia

5.4. Gestión del riesgo

Durante el desarrollo del proyecto pueden surgir obstáculos que afecten a su progreso. Por ello, en la planificación se ha reservado un margen adicional de horas en la fase de desarrollo técnico, y se han definido planes de contingencia concretos para cada riesgo identificado. Los posibles riesgos se han descrito en la Sección 3.4. A continuación, se indica su nivel de impacto en el proyecto y las alternativas previstas para mitigarlos.

R.1 – Complejidad de traducción visual → IaC *Probabilidad: alta / Impacto: alto*

Dificultad para definir una abstracción común entre el modelo visual y los dialectos IaC de distintos proveedores (Terraform HCL, Pulumi).

Plan B:

- **Tarea alternativa:** Reducir DT.13 (Biblioteca de recursos Cloud) limitándola a un subconjunto mínimo de recursos AWS con plantillas HCL fijas, eliminando el soporte multi-proveedor del MVP y documentándolo como trabajo futuro.
- **Impacto en duración:** DT.13 se reduce de 25 h a 15 h, liberando 10 h que se redistribuyen en DT.10 (+5 h para consolidar las pruebas de traducción) y DT.16 (+5 h para el generador). La duración total del proyecto no se ve afectada.
- **Recursos adicionales:** Ninguno. La reducción de alcance compensa el esfuerzo sin requerir herramientas ni tiempo extra.

R.2 – Curva de aprendizaje de Rust *Probabilidad: media / Impacto: medio*

El aprendizaje de Rust (ownership, borrowing, lifetimes) puede retrasar el inicio de la implementación técnica si la curva resulta más pronunciada de lo estimado.

Plan B:

- **Tarea alternativa:** Ampliar DT.5 (Aprendizaje de Rust, Tauri y React) en 10 h adicionales, compensadas reduciendo DT.9 (Canvas básico) en 5 h, simplificando la UI inicial a un prototipo de baja fidelidad sin drag-and-drop.
- **Impacto en duración:** El Sprint 2 se alarga en 5 h netas, absorbibles dentro del margen de contingencia reservado. La fecha de entrega final no se ve comprometida.
- **Recursos adicionales:** Acceso a documentación oficial de Rust (*The Rust Programming Language*) y recursos online gratuitos (R1 existente). No se requieren recursos materiales adicionales.

R.3 – Estimaciones insuficientes por imprevistos *Probabilidad: media / Impacto: medio*

Imprevistos personales o académicos pueden reducir la disponibilidad en semanas clave del desarrollo.

Plan B:

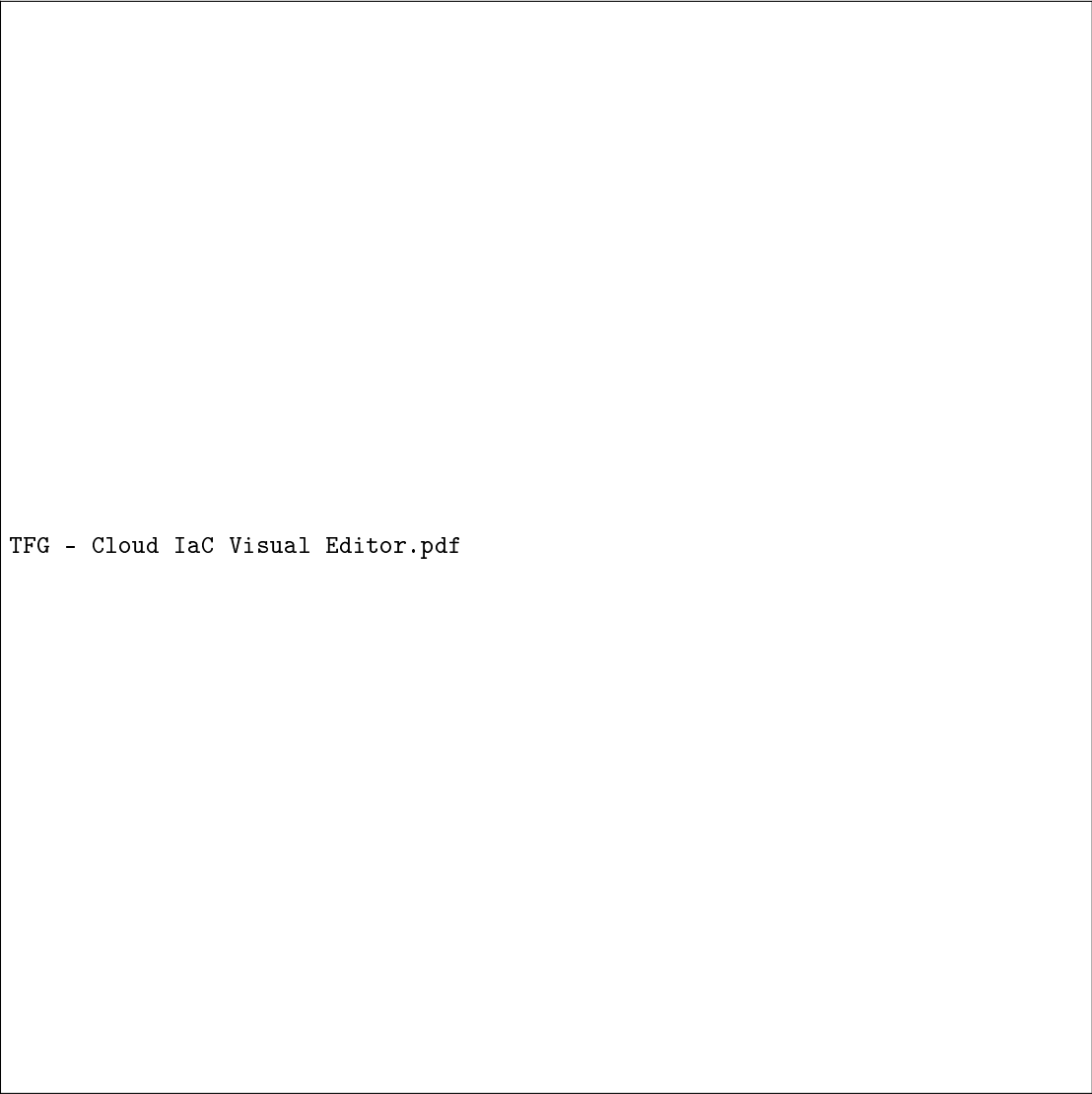
- **Tarea alternativa:** Activar el margen de contingencia del Sprint 6, diseñado explícitamente para absorber retrasos de hasta una semana. Si el retraso supera ese margen, se reducirá el alcance de DT.21 (Preparación de la defensa) de 15 h a 8 h, limitando la demo del prototipo a un caso de uso representativo.

- **Impacto en duración:** Un retraso de hasta 7 días se absorbe sin modificar la fecha de entrega del 15 de junio. Un retraso mayor obligaría a negociar una prórroga con el director, impactando únicamente el Sprint 6.
- **Recursos adicionales:** Ninguno. La gestión del riesgo se basa en reordenación de prioridades, no en recursos extra.

La distribución temporal de todas las tareas, incluyendo los márgenes de contingencia descritos, se visualiza en el diagrama de Gantt de la sección siguiente.

5.5. Diagrama de Gantt

El siguiente diagrama de Gantt refleja la planificación descrita en las secciones anteriores, mostrando la secuencia de sprints, las dependencias entre tareas y el margen reservado para la gestión de riesgos.



TFG - Cloud IaC Visual Editor.pdf

Figura 1: Diagrama de Gantt
Fuente: elaboración propia con ProjectLibre

6. Gestión económica

En esta sección se identifican, estiman y cuantifican los costes del proyecto. Siguiendo la estructura recomendada, el presupuesto se divide en: *Costes de Personal por Actividad (CPA)*, *Costes Genéricos (CG)*, *Contingencias* y *Costes de Imprevistos*. Todos los costes de personal se derivan directamente de las tareas y horas definidas en el diagrama de Gantt (Sección ??), garantizando la coherencia entre la planificación temporal y el presupuesto.

6.1. Costes de Personal por Actividad (CPA)

6.1.1. Recursos humanos

Para calcular el coste de los recursos humanos, se han asignado tarifas estándar de mercado a los tres roles definidos en el proyecto, consultadas a través del portal de empleo [31]. Al salario bruto estimado por hora se le ha aplicado un factor multiplicador de **1,3** para incluir los costes asociados a la Seguridad Social (aproximadamente un 30 % del salario bruto, correspondiente a las cotizaciones empresariales en España).

Rol	Salario bruto (€/h)	Coste con SS (€/h)
Jefe de proyecto (JP)	24,00	31,20
Ingeniero de software (IS)	17,00	22,10
Redactor técnico (RT)	14,00	18,20

Cuadro 3: Tarifas horarias de los recursos humanos del proyecto

Fuente: elaboración propia a partir del portal de empleo citado.

Cuando una tarea es compartida por dos roles, el coste se calcula como la media aritmética de los costes horarios de ambos roles, multiplicada por el número de horas de la tarea. Por ejemplo, para una tarea de 10 h compartida entre JP (31,20 €/h) y RT (18,20 €/h):

$$\frac{31,20 + 18,20}{2} \times 10 = 24,70 \times 10 = 247,00 \text{€}$$

Las tareas y sus costes se derivan directamente de la planificación del Gantt (Sección ??). Para cada tarea se indica el rol responsable según lo definido en la Tabla ??, las horas estimadas y el coste total:

Sprint	Tarea	Rol	Horas	Coste (€)
GEP	Gestión del Proyecto	JP, RT	90	2.223,00
Sprint 1	DT.1 Investigación y estado del arte	IS	15	331,50
	DT.2 Definición de requisitos	JP, IS	25	666,25
	DT.3 Selección tecnológica	IS	10	221,00
	DT.4 Documentación sprint	JP, RT	10	247,00
Sprint 2	DT.5 Aprendizaje tecnologías	IS	30	663,00
	DT.6 Diseño arquitectura	IS	15	331,50
	DT.7 Configuración entorno	IS	10	221,00
	DT.8 Documentación sprint	JP, RT	10	247,00
Sprint 3	DT.9 Canvas básico	IS	45	994,50
	DT.10 Pruebas traducción IaC	IS	15	331,50
	DT.11 Documentación sprint	JP, RT	10	247,00
Sprint 4	DT.12 Editor visual	IS	30	663,00
	DT.13 Biblioteca cloud	IS	25	552,50
	DT.14 Gestión conexiones	IS	25	552,50
	DT.15 Documentación sprint	JP, RT	10	247,00
Sprint 5	DT.16 Generador Terraform	IS	25	552,50
	DT.17 Integración sistema	IS	25	552,50
	DT.18 Validación seguridad	IS	25	552,50
	DT.19 Documentación sprint	JP, RT	10	247,00
Sprint 6	DT.20 Redacción memoria	RT	55	1.001,00
	DT.21 Preparación defensa	JP, RT	15	370,50
	DT.22 Seguimiento sprint	JP, RT	10	247,00
Coste total de personal (CPA)				12.262,25 €

Cuadro 4: Coste estimado de recursos humanos por tarea y sprint

El coste por hora incluye el incremento del 30 % por cotizaciones a la Seguridad Social. Las tareas compartidas se calculan como la media de los costes horarios de los roles implicados. Fuente: elaboración propia a partir del Gantt (Sección ??).

6.2. Costes Genéricos (CG)

Los costes genéricos son aquellos que no dependen directamente del número de tareas realizadas, sino de factores estructurales del proyecto: hardware, software, espacio de trabajo y electricidad. Se han estimado considerando la duración total del proyecto de **540 horas** (450 h de desarrollo técnico y 90 h de gestión), distribuidas a lo largo de **5 meses** (febrero–junio 2026).

6.2.1. Hardware

En un entorno profesional, el hardware sería compartido entre varios miembros del equipo. En este proyecto, sin embargo, todos los roles son desempeñados por una única persona, por lo que se contabiliza un único puesto de trabajo. El coste imputable al proyecto no es el precio de adquisición completo, sino la parte proporcional correspondiente a las horas de uso durante el proyecto, calculada mediante amortización lineal.

Dispositivo	Coste de adquisición (€)
Torre PC (AMD Ryzen 7 5800X, RTX 3060, 32 GB RAM, 1 TB NVMe)	1.300,00
Monitor HP Omen	125,10
Monitor Dell	110,00
Teclado	23,99
Ratón	5,99
Total adquisición	1.565,08 €

Cuadro 5: Coste de adquisición del hardware utilizado
Fuente: elaboración propia.

Amortización: Se considera una vida útil media de **6 años**, criterio habitual para equipos informáticos en entornos profesionales. Con 251 días laborables al año y 8 horas por jornada, el total de horas de vida útil es:

$$251 \times 8 \times 6 = 12,048 \text{ h}$$

El coste amortizado por hora de uso es:

$$\frac{1,565,08}{12,048} = 0,1299 \text{ €/h}$$

Para las 540 horas del proyecto:

$$0,1299 \times 540 = \mathbf{70,15 \text{ €}}$$

6.2.2. Software

Todas las herramientas empleadas (Rust, Tauri, React, GitHub, Trello, $\text{\LaTeX}$, VS Code) son gratuitas y de código abierto, por lo que el coste de licencias es **0,00 €**. Esta decisión es coherente con el enfoque open source del proyecto y elimina cualquier dependencia de software propietario.

6.2.3. Espacio de trabajo

El proyecto se desarrolla en el domicilio del autor. Para estimar el coste imputable al proyecto se consideran dos componentes: el alquiler mensual proporcional y otros gastos generales fijos durante la duración del proyecto.

- **Alquiler y suministros:** $2.030 \text{ €/mes} \times 5 \text{ meses} = 10.150 \text{ €}$
- **Otros gastos generales** (limpieza, material de oficina, etc.): 2.000 € fijos para toda la duración

$$2,030 \times 5 + 2,000 = \mathbf{12,150 \text{ €}}$$

6.2.4. Electricidad

El coste de electricidad se calcula en dos partes: el consumo de los dispositivos del puesto de trabajo durante las 540 horas del proyecto, y el consumo general del domicilio durante los 5 meses.

Para el consumo de dispositivos, se estima la potencia media de cada equipo y se multiplica por las 540 horas totales del proyecto:

Dispositivo	Potencia (W)	Horas uso	Consumo (kWh)
Torre PC	250	540	135,00
Monitor HP Omen	35	540	18,90
Monitor Dell	30	540	16,20
Teclado	5	540	2,70
Ratón	3	540	1,62
Total consumo dispositivos			174,42 kWh

Cuadro 6: Consumo eléctrico estimado de los dispositivos del proyecto
Fuente: elaboración propia.

Según los datos oficiales del precio medio de la electricidad en España [32], el coste medio es de **0,1975 €/kWh**. El coste de los dispositivos es:

$$174,42 \times 0,1975 \approx 34,45 \text{ €}$$

A esto se añade el consumo proporcional del domicilio durante los 5 meses del proyecto, estimado en 60 €/mes basándose en la factura media de un hogar español con equipos ofimáticos:

$$60 \times 5 = 300 \text{ €}$$

$$34,45 + 300 \approx \mathbf{334,45 \text{ €}}$$

6.2.5. Resumen de costes genéricos

Concepto	Coste (€)
Hardware (amortización)	70,15
Software	0,00
Espacio de trabajo	12.150,00
Electricidad	334,45
Total costes genéricos (CG)	12.554,60

Cuadro 7: Resumen de costes genéricos del proyecto
Fuente: elaboración propia.

6.3. Contingencia

En proyectos informáticos es habitual reservar un margen de contingencia para cubrir desviaciones en los costes estimados. Para este proyecto se aplica un **15 %** sobre la suma de CPA y CG, porcentaje estándar en proyectos de desarrollo software de alcance medio. Este margen cubre posibles incrementos en horas de trabajo debidos a imprevistos técnicos no contemplados en los riesgos identificados.

$$\text{Contingencia} = (CPA + CG) \times 0,15 = (12,262,25 + 12,554,60) \times 0,15 \approx \mathbf{3,722,53 \text{ €}}$$

6.4. Costes de Imprevistos

Los imprevistos se derivan directamente de los riesgos identificados en la Sección ???. Siguiendo la metodología GEP, cada imprevisto no se costea al 100 % sino según la probabilidad estimada de que ocurra el riesgo: para un riesgo con probabilidad p , se imputa $p \times \text{coste_plan_B}$. Esto configura un sistema de compensación: si solo ocurre uno de los imprevistos, lo presupuestado por los que no ocurren cubre parcialmente el exceso del que sí ocurre.

Riesgo	Plan B	Prob.	Horas	Coste plan B (€)	Imprevisto (€)
R.1	Reducción DT.13, re-distribución 10 h IS	70 %	10 h IS	221,00	154,70
R.2	Ampliación DT.5 en 5 h netas IS	40 %	5 h IS	110,50	44,20
R.3	Reducción DT.21 en 7 h JP/RT	40 %	7 h JP/RT	172,90	69,16
Total					268,06

Cuadro 8: Estimación de costes de imprevistos del proyecto
Fuente: elaboración propia a partir de la gestión de riesgos (Sección ???).

6.5. Presupuesto Total

La siguiente tabla resume el presupuesto total del proyecto, integrando todas las partidas calculadas:

Concepto	Importe (€)
Costes de Personal por Actividad (CPA)	12.262,25
Costes Genéricos (CG)	12.554,60
Subtotal (CPA + CG)	24.816,85
Contingencias (15 % sobre subtotal)	3.722,53
Imprevistos	268,06
Presupuesto total estimado	28.807,44

Cuadro 9: Resumen del presupuesto total estimado del proyecto
Fuente: elaboración propia.

El presupuesto total estimado asciende a **28.807,44 €**. Las partidas dominantes son los costes de personal (42,6 % del total) y el espacio de trabajo (42,2 %), lo que es coherente con un proyecto de desarrollo software individual realizado desde el domicilio. Los imprevistos representan únicamente el 0,9 % del total, reflejo de que los planes B definidos priorizan la reorganización de tareas existentes sobre la incorporación de horas adicionales. En el contexto académico de un TFG, donde el estudiante no percibe remuneración, el coste efectivo real se limita a los costes genéricos (**12.554,60 €**), aunque el presupuesto refleja el valor económico real si el proyecto se desarrollara en un entorno profesional.

6.6. Control de Gestión

El objetivo del control de gestión es comparar y evaluar las desviaciones entre el presupuesto planificado y los costes reales en que va incurriendo el proyecto a medida que avanza. Siguiendo la metodología del GEP, el control se articula en torno a tres preguntas fundamentales:

- **¿Dónde** se ha producido la desviación? (en qué tarea o sprint)
- **¿Por qué** se ha producido? (desviación en tarifa o en eficiencia)
- **¿De qué cuantía** es la desviación?

Para responder a estas preguntas se definen los siguientes indicadores numéricos, que se revisarán al final de cada sprint coincidiendo con la reunión de seguimiento con el director:

Desvío en tarifa (precio)

Mide si el coste real por hora difiere del coste estimado, manteniendo constante el consumo de horas:

$$D_{tarifa} = (C_{h,est} - C_{h,real}) \times H_{real}$$

donde $C_{h,est}$ es el coste horario estimado del rol, $C_{h,real}$ el coste horario real y H_{real} las horas realmente consumidas. En este proyecto, al ser el mismo autor quien desempeña todos los roles, este desvío sería nulo salvo que se requiriera subcontratar trabajo externo.

Desvío en eficiencia (consumo)

Mide si se han consumido más o menos horas de las planificadas para una tarea, aplicando el coste estimado como referencia:

$$D_{eficiencia} = (H_{est} - H_{real}) \times C_{h,est}$$

donde H_{est} son las horas planificadas para la tarea. Un valor negativo indica sobreconsumo de horas. Este es el indicador principal del proyecto, ya que el principal riesgo de desviación es la infraestimación de horas de desarrollo.

Desvío total de mano de obra por sprint

Integra ambos efectos para cada sprint k :

$$D_{total,k} = \sum_{i \in Sprint_k} [(C_{h,est,i} - C_{h,real,i}) \times H_{real,i} + (H_{est,i} - H_{real,i}) \times C_{h,est,i}]$$

La siguiente tabla muestra los valores de referencia planificados por sprint, que servirán como línea base para el cálculo de los desvíos reales:

Sprint	Horas planif.	Coste planif. (€)	Coste acum. (€)	Umbral alerta (5 %)
GEP	90	2.223,00	2.223,00	111,15
Sprint 1	60	1.465,75	3.688,75	73,29
Sprint 2	65	1.462,50	5.151,25	73,13
Sprint 3	70	1.573,00	6.724,25	78,65
Sprint 4	90	2.015,00	8.739,25	100,75
Sprint 5	85	1.904,50	10.643,75	95,23
Sprint 6	80	1.618,50	12.262,25	80,93
Total	540	12.262,25		

Cuadro 10: Línea base del presupuesto por sprint para el control de gestión

Fuente: elaboración propia a partir de la Tabla ??.

Si al final de un sprint el $D_{total,k}$ supera el **5 %** del coste planificado del sprint (columna “Umbral alerta”), se activará el siguiente protocolo de actuación en tres niveles:

1. **Nivel 1 — Revisión de alcance:** reducir el alcance de las tareas pendientes del sprint siguiente para compensar el sobrecoste.
2. **Nivel 2 — Imprevistos:** si la desviación acumulada supera la partida de imprevistos (268,06 €), se activará el fondo de contingencia (3.722,53 €).
3. **Nivel 3 — Renegociación:** si el coste total proyectado supera el presupuesto total más contingencias, se negociará con el director una reducción formal del alcance del MVP.

Este sistema implementa el concepto de **presupuesto flexible** descrito en la metodología GEP: el presupuesto pactado es uno, pero a partir del control por sprint pueden generarse estimaciones actualizadas del coste final del proyecto, permitiendo anticipar desviaciones antes de que sean irrecuperables.

7. Informe de Sostenibilidad

Se evalúan los ámbitos ambiental, económico y social del proyecto, incluyendo una autoevaluación de competencias en sostenibilidad.

7.1. Autoevaluación

Tras completar el cuestionario EDINSOST de autoevaluación de competencias en sostenibilidad, he identificado puntos fuertes y áreas de mejora en los tres ámbitos de la sostenibilidad: ambiental, social y económica. En ingeniería se tiende a priorizar la dimensión tecnológica y económica, relegando el impacto ambiental y social. El cuestionario me ha permitido incorporar conceptos que no tenía plenamente asimilados, especialmente en economía circular, economía social y economía ecológica.

En cuanto a mis puntos fuertes, tengo un conocimiento razonable de la sostenibilidad ambiental en el sector tecnológico, especialmente en consumo energético de infraestructuras Cloud y eficiencia computacional. Soy consciente de que las TIC contribuyen significativamente a las emisiones globales de CO₂ y de que soluciones más eficientes generan un impacto ambiental positivo. Además, mi formación en gestión económica me permite estimar costes y valorar la viabilidad de un proyecto con cierta solvencia.

Respecto a las competencias adquiridas, he aprendido a considerar la huella ecológica de productos a lo largo de todo su ciclo de vida, aplicando estrategias de reducción, reutilización y reciclaje. En el ámbito social, he reflexionado sobre el impacto de mis proyectos en la salud, seguridad y justicia social, incluyendo accesibilidad, diversidad y equidad.

En cuanto a las áreas de mejora, mi conocimiento sobre métricas cuantitativas de sostenibilidad, como el análisis del ciclo de vida o las metodologías de huella de carbono, es aún limitado. Tampoco he trabajado con marcos formales de sostenibilidad social, aunque comprendo sus principios fundamentales.

En conclusión, la autoevaluación ha reforzado la necesidad de integrar la sostenibilidad de forma transversal desde las etapas más tempranas del proyecto, y reconozco que debo seguir profundizando en estas competencias para un análisis más avanzado.

7.1.1. Dimensión Económica

En relación con el Proyecto Puesto en Producción (PPP), el coste total estimado asciende a **28.807,44 €**, como se muestra en la Tabla ???. Este importe incluye los Costes de Personal por Actividad (CPA), los Costes Genéricos (CG), un margen de contingencia del 15 % y los costes de imprevistos, calculados según la probabilidad estimada de materialización de cada riesgo identificado. En el contexto académico individual, el coste efectivo se limita a los recursos materiales y amortizaciones (**12.554,60 €**), aunque el presupuesto refleja el valor económico real de los recursos implicados.

Respecto al estado del arte, herramientas comerciales de modelado visual de IaC como *Brainboard* tienen costes entre 100 € y 500 € mensuales por usuario. La solución propuesta, al ser de código abierto, elimina estos costes y supone una alternativa económica relevante para equipos pequeños, startups y entornos educativos. Además, su interfaz visual reduce la curva de aprendizaje de IaC, disminuyendo el tiempo de incorporación de nuevos perfiles técnicos y generando un ahorro indirecto en formación y productividad.

7.1.2. Dimensión Ambiental

Impacto de la realización (PPP): El impacto ambiental directo de la ejecución de este TFG es reducido y se limita principalmente al consumo eléctrico de los dispositivos durante las 540 horas de trabajo, estimado en **34,45 €**. Además, no se ha adquirido nuevo hardware, reutilizando equipos existentes y minimizando el impacto asociado a su fabricación y transporte.

Minimización del impacto (PPP): Se ha evitado el uso de servidores físicos dedicados, trabajando en entornos locales con herramientas de software libre. El desarrollo individual en un único puesto de trabajo implica un coste de **70,15 €** por amortización de hardware y **334,45 €** en consumo eléctrico total, sin necesidad de infraestructura Cloud durante esta fase.

Vida útil y mejora de la solución: La solución presenta un potencial impacto ambiental positivo en fase operativa. Su interfaz visual facilita la gestión eficiente de infraestructuras Cloud, ayudando a evitar sobreaprovisionamientos y a identificar recursos ociosos, lo que favorece prácticas de eficiencia energética en centros de datos.

7.1.3. Dimensión Social

Aporte a nivel personal (PPP): La realización de este proyecto supone un crecimiento profesional significativo al asumir todos los roles del proyecto (jefe de proyecto, ingeniero de software y redactor técnico) durante 540 horas. Esto ha implicado aplicar metodologías ágiles como Scrum y Kanban, trabajar con tecnologías como Rust y Node.js, y consolidar competencias técnicas junto con habilidades de autogestión.

Estado del arte y mejora social: El acceso a herramientas visuales de IaC suele estar limitado a soluciones comerciales o a perfiles con experiencia avanzada en Terraform o Pulumi, generando barreras para estudiantes, perfiles junior y pequeñas organizaciones. La solución propuesta, al ser de código abierto y visual, reduce estas barreras y favorece la incorporación de nuevos perfiles al ámbito del Cloud Computing.

Necesidad real del proyecto: Los equipos Cloud afrontan problemas de interoperabilidad y complejidad entre plataformas como AWS, GCP y Azure, especialmente para perfiles con menor experiencia. La herramienta propuesta ofrece una alternativa accesible e interoperable alineada con la filosofía Open Source, contribuyendo a reducir la brecha tecnológica.

8. Arquitectura del sistema

Esta sección describe la arquitectura general de la solución desde una perspectiva de visión global. El objetivo es mostrar cómo las decisiones tomadas para poder cumplir con los objetivos y requisitos definidos en la (Sección ??)

8.1. De los requisitos a la arquitectura

Los subobjetivos definidos en la Sección ?? imponen varias restricciones sobre la forma del sistema. Se requiere una interfaz gráfica capaz de representar componentes Cloud reutilizables y conectables; un mecanismo que traduzca de forma fiable ese diseño visual a código de **Infraestructura como Código (IaC)**; una arquitectura modular y extensible que permita incorporar nuevos proveedores en el futuro; y, como requisitos transversales, un enfoque **open source** y un coste de adopción reducido para el usuario final.

Estas restricciones descartan algunas opciones como: una aplicación puramente web, alojada en un servidor remoto, introduciría una dependencia de infraestructura externa y complicaría tanto la instalación local como la edición de ficheros del usuario en disco. Una aplicación de escritorio en lenguaje propietario, comprometería la portabilidad y la reutilización del código. La opción que ha resultado coherente con el conjunto de requisitos ha sido la de una **aplicación de escritorio multiplataforma**, que opera íntegramente en local y se apoya en herramientas estándar del ecosistema Cloud para su funcionamiento.

8.2. Visión global del sistema

El sistema se organiza en torno a cuatro bloques principales que colaboran mediante interfaces estables: una **capa de presentación**, una **capa de aplicación**, una **capa de infraestructura** y una **pipeline de extracción de recursos**. Esta organización permite separar la interacción con el usuario, la lógica del dominio, la integración con el entorno local y la obtención del catálogo de recursos. El núcleo semántico que vertebra estos bloques es el **modelo de dominio**, cuya estructura se detalla en la Sección ??.

La **pipeline de extracción de recursos** obtiene el catálogo de recursos disponibles para cada proveedor cloud a partir de la información expuesta por Terraform. Esta pieza garantiza la **coherencia** y la **actualización** del catálogo, ya que se apoya en la fuente de verdad oficial del proveedor y evita mantener descripciones manuales susceptibles de desfase.



Figura 2: Pipeline de extracción de recursos
Fuente: elaboración propia

La **capa de presentación** es responsable de la interacción con el usuario: presenta el lienzo, el catálogo de recursos, los formularios de edición y los mecanismos de navegación. Está construida sobre un stack web moderno, lo que facilita el desarrollo mediante componentes reutilizables y permite una posible evolución futura hacia una versión web sin reescritura sustancial de esta capa.

La **capa de aplicación** concentra la lógica funcional del sistema: gestiona el estado del proyecto, valida operaciones, coordina los distintos componentes y traduce el diseño interno a código **Infraestructura como Código (IaC)**. En esta capa reside la lógica de negocio que determina cómo se crean, modifican y serializan los elementos del sistema.

Por último, la **capa de infraestructura** conecta la aplicación con el entorno local. Se encarga de invocar Terraform como subprocesso, gestionar las credenciales del proveedor cloud y acceder al sistema de ficheros del proyecto. Esta capa reduce el acoplamiento con herramientas externas y mejora la **portabilidad** de la solución.

Para situar el sistema en su entorno y mostrar la relación entre sus bloques internos se emplea la notación **C4** [33] en dos niveles complementarios. El **nivel 1 (contexto)** sitúa la aplicación frente

al usuario y los sistemas externos con los que interactúa, mientras que el **nivel 2 (contenedores)** descompone la aplicación en sus bloques internos y detalla las tecnologías asociadas a cada uno.

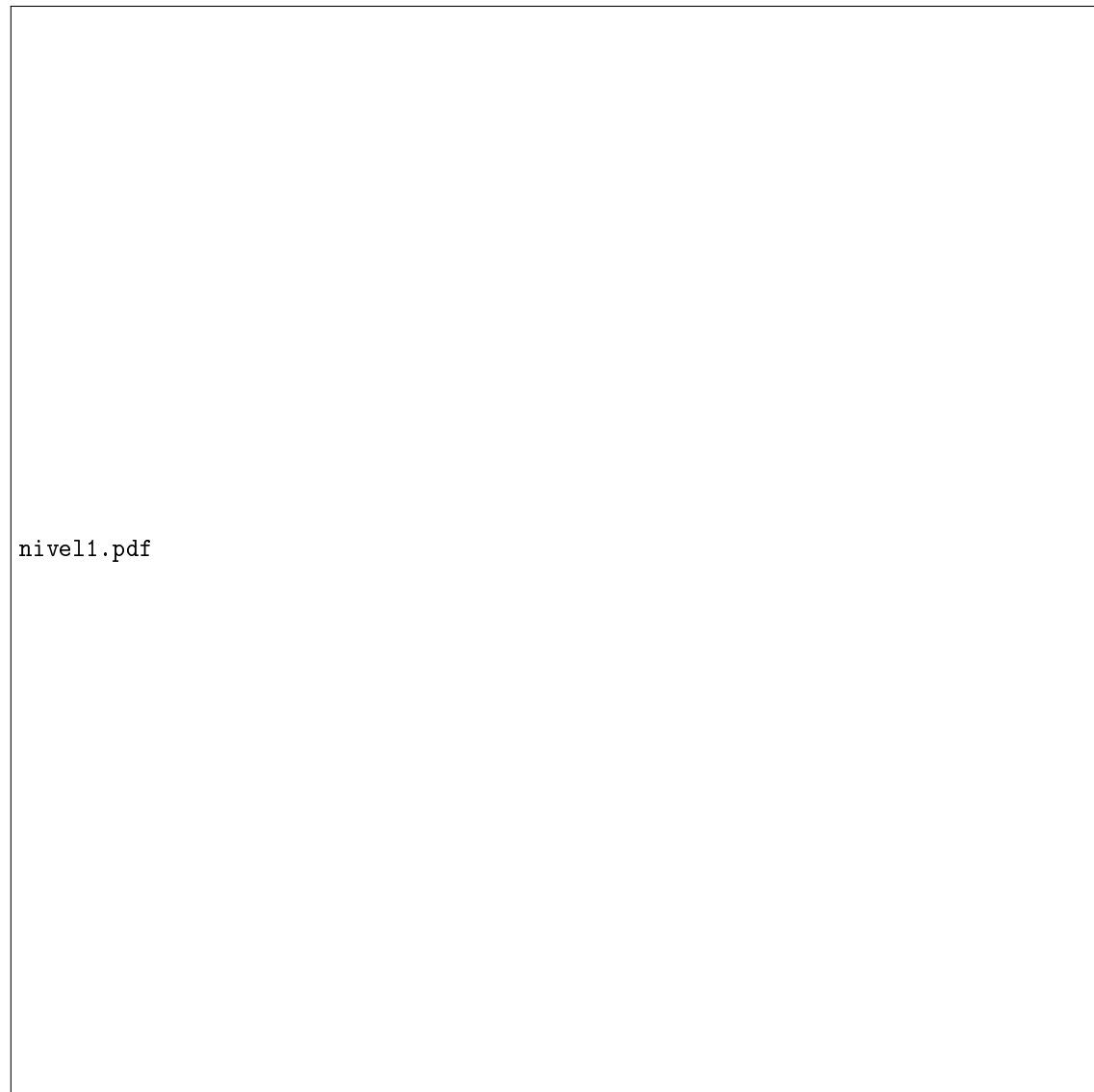


Figura 3: Vista de contexto del sistema (C4, nivel 1)
Fuente: elaboración propia

8.3. Stack tecnológico

El stack tecnológico se ha seleccionado de forma que cada decisión responda a un requisito concreto de los descritos en la Sección ??: ejecución **multiplataforma** y **local**, arquitectura **modular y extensible**, y compromiso con un enfoque **open source**. A continuación se justifica la elección de cada pieza.

Tauri como contenedor de aplicación de escritorio

Para empaquetar la aplicación como ejecutable de escritorio se ha optado por **Tauri** frente a alternativas como Electron [34]. Tauri utiliza el motor de renderizado nativo del sistema operativo en lugar de embeber un Chromium completo, lo que reduce drásticamente el tamaño del binario y el consumo de memoria respecto a un equivalente en Electron. Además, su backend nativo en Rust permite invocar subprocesos del sistema operativo de forma controlada, condición necesaria para integrar la CLI de Terraform y la terminal embebida descritas en la capa de infraestructura.

Rust en el backend nativo

El backend nativo, responsable de la capa de infraestructura, se ha implementado en **Rust**. La elección se sostiene en tres motivos: la seguridad de memoria que ofrece su sistema de *ownership* y *borrowing*, lo que elimina toda una clase de errores en código que gestiona procesos, ficheros y credenciales; el rendimiento sostenido al lanzar y parsear en *streaming* la salida de Terraform, que puede ser voluminosa; y la robustez de su ecosistema para concurrencia (lectores de `stdout/stderr` en paralelo) y manejo de procesos interactivos, requerida para la confirmación de `apply` y `destroy`.

React con TypeScript en la capa de presentación

La interfaz se construye con **React** y **TypeScript**. React aporta un modelo de componentes reutilizables idóneo para una UI con paneles independientes (catálogo, lienzo, inspector, terminal) y dispone de un ecosistema maduro de bibliotecas para canvas, *drag-and-drop* y edición visual. TypeScript añade tipado estático al contrato entre el frontend y los comandos Tauri del backend, lo que detecta inconsistencias en tiempo de compilación y mejora la mantenibilidad del código a medio plazo.

Go en la pipeline de extracción de recursos

La pipeline descrita en la Sección ?? se ha implementado en **Go** [35]. La razón principal es práctica: HashiCorp mantiene una biblioteca oficial en Go para interpretar el schema de los proveedores de Terraform, lo que evita reimplementar el parser. Además, Go es especialmente adecuado para herramientas de procesamiento masivo de ficheros JSON [**noauthor _json _nodate**], que es exactamente la naturaleza del problema. Es importante recalcar que la pipeline es un **preproceso offline**: produce ficheros estáticos que pasan a formar parte de la distribución de la aplicación, por lo que el usuario final no necesita tener Go instalado ni ejecutar la pipeline.

En conjunto, todas las piezas del stack son **open source y gratuitas**, lo que es coherente con el subobjetivo de licenciamiento abierto y elimina cualquier dependencia con software propietario.

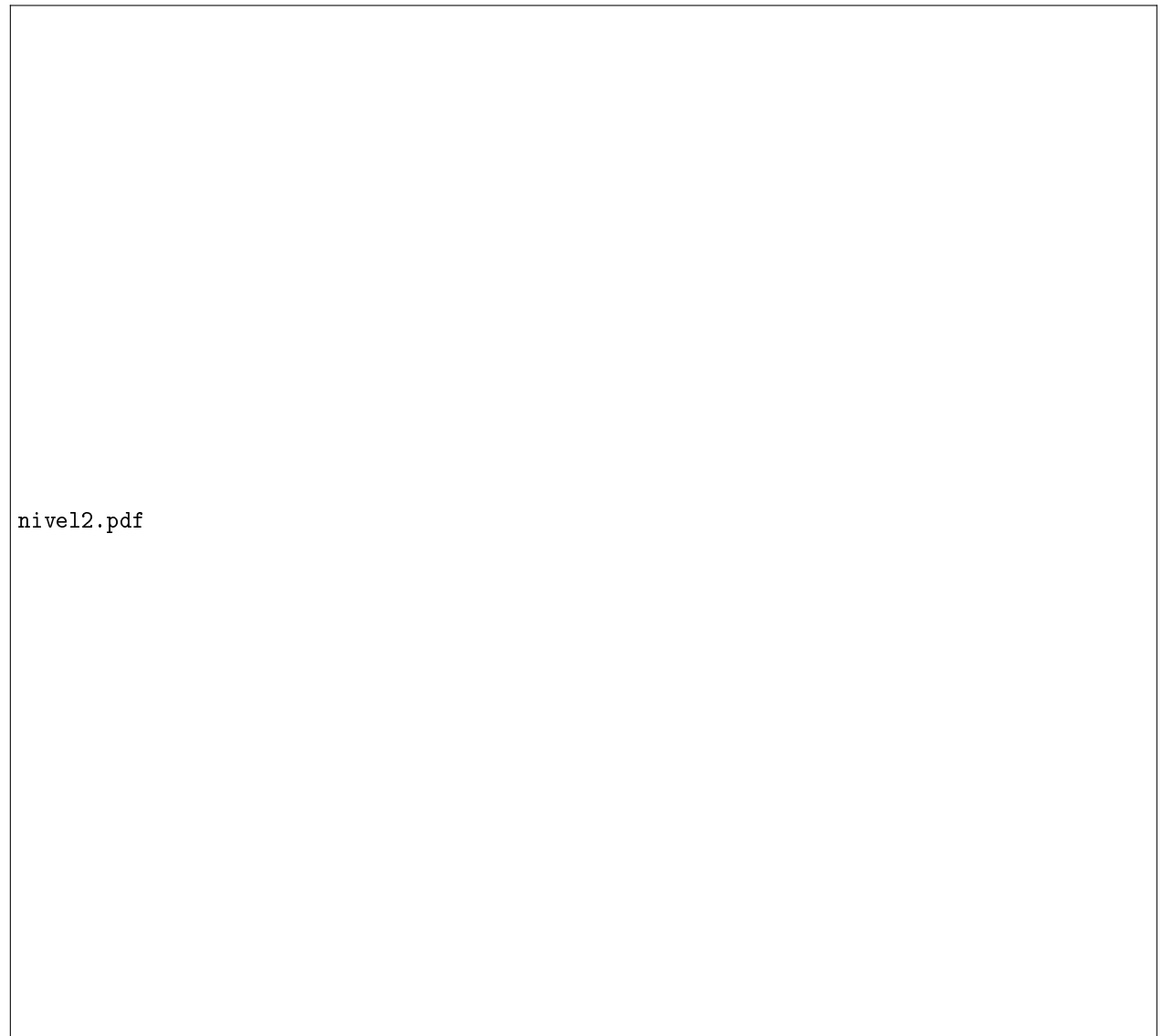


Figura 4: Vista de contenedores del sistema (C4, nivel 2)
Fuente: elaboración propia

8.4. Modelo de dominio

Una decisión central de la arquitectura es situar el **modelo de dominio** como fuente de verdad única del sistema. Mientras que el **modelo de presentación** describe el estado visual, posiciones, selecciones, atributos gráficos. El modelo de dominio captura la semántica real de la infraestructura: qué recursos existen, cómo se relacionan y qué valores tienen sus propiedades, con independencia de cómo se representen visualmente o del formato IaC al que se traduzcan. Su estructura interna, ciclo de vida y esquema de persistencia se desarrollan en detalle en la Sección ??.

8.5. Síntesis

En conjunto, la arquitectura propuesta separa de forma explícita la representación visual, la lógica de aplicación, la integración con infraestructura y la obtención del catálogo de recursos. El sistema se apoya en un **modelo de dominio** único como fuente de verdad, mientras que la capa de presentación actúa como proyección visual de ese estado, la capa de aplicación concentra las operaciones necesarias para modificarlo, validarlo y traducirlo a **Infraestructura como Código (IaC)** y la capa de infraestructura se encarga de la interacción con el entorno local y las herramientas externas.

Esta separación reduce el acoplamiento entre interfaz y lógica, mejora la mantenibilidad y facilita la evolución del sistema. Además, el uso de un **catálogo extensible** permite mantener la neutralidad respecto al proveedor cloud y abre la puerta a incorporar nuevos entornos sin modificar el núcleo funcional de la aplicación.

9. Casos de uso

Una vez descrita la arquitectura, esta sección resume las funcionalidades que la aplicación pone a disposición del usuario. El actor principal es el **Usuario**, ya caracterizado en la Sección ???. Los **sistemas externos** que intervienen en estos flujos (*Terraform CLI* y *Proveedor cloud*) son los mismos que aparecen en el diagrama de contexto de la Figura ??.

9.1. Visión general de las funcionalidades

La aplicación cubre el ciclo de vida completo de una arquitectura cloud, desde su diseño inicial hasta la comparación entre lo diseñado y lo realmente desplegado. Las funcionalidades pueden agruparse en cuatro bloques complementarios.

El primer bloque corresponde a la **gestión de proyectos y vistas**: el usuario puede crear, abrir y guardar proyectos, así como organizar cada proyecto en una o varias vistas (típicamente *dev*, *staging* o *prod*), cada una con su propio estado y sus propios archivos. El segundo bloque agrupa las acciones de **diseño visual**: arrastrar recursos desde el catálogo al lienzo, editar sus atributos en el inspector de propiedades y conectarlos entre sí mediante *enlaces* que generan automáticamente las referencias HCL correspondientes. Como complemento, el usuario también puede **editar el HCL manualmente** desde el editor embebido: el sistema detecta los cambios y los reconcilia con el modelo de dominio, manteniendo coherentes el lienzo, el inspector y el código.

El tercer bloque comprende las acciones de **despliegue**, que delegan en la *Terraform CLI* a través de la capa de infraestructura. El usuario puede ejecutar **plan** para previsualizar los cambios, **apply** para materializarlos en el proveedor cloud, y **destroy** para revertir el despliegue. La salida del subproceso se transmite en tiempo real y, en el caso de **apply**, la confirmación interactiva se intercepta y se envía desde la propia interfaz, sin requerir que el usuario abandone la aplicación. El cuarto bloque cubre las funcionalidades de **observación**: la vista *Diff* proyecta sobre el lienzo los cambios previstos por el último **plan**, mientras que la vista *Cloud* consulta el estado real de la infraestructura desplegada, permitiendo identificar de un vistazo cualquier divergencia entre diseño y realidad.

10. Pipeline de extracción de recursos

Esta sección describe cómo se obtienen los recursos de un proveedor cloud. Aunque el desarrollo se ha realizado sobre el proveedor de **Amazon Web Services** para acotar el alcance del MVP, el proceso aquí descrito es genérico: el mismo pipeline aplicado sobre Google Cloud o Microsoft Azure produciría un catálogo equivalente sin requerir cambios en la aplicación que lo consume.

10.1. Motivación: ¿por qué no codificar el catálogo a mano?

Una primera aproximación al problema, sería escribir manualmente la lista de recursos cloud que la aplicación soporta, junto con sus atributos y las plantillas de código IaC asociadas. Este enfoque tiene tres problemas fundamentales que justifican la inversión en un pipeline automatizado.

En primer lugar, el catálogo de un proveedor cloud mayor es extraordinariamente amplio: el provider de AWS para Terraform expone aproximadamente **1.500 recursos** y **600 fuentes de datos**. Codificarlos manualmente no es viable en el marco temporal de un Trabajo Final de Grado, ni siquiera para un subconjunto razonable.

En segundo lugar, esa información cambia con cada versión del proveedor: nuevas funcionalidades introducen nuevos atributos, otras quedan marcadas como obsoletas, y los tipos de algunos campos se refinan. Mantener un catálogo manual sincronizado con esa evolución sería una tarea de **mantenimiento** continuado y propensa a errores.

En tercer lugar, la documentación pública del proveedor está pensada para ser leída, no parseada, y omite información estructural relevante de los atributos.

La solución natural es apoyarse en la propia herramienta IaC. Terraform expone, mediante un comando de su CLI, una representación completa del schema del proveedor en formato JSON. Esa representación procede directamente del binario oficial del proveedor lo que garantiza que sea **fidedigna** (está siempre alineada con la versión del proveedor que se está utilizando). Como se observa en la Figura ??, el proceso completo de la pipeline transforma el schema del proveedor en los artefactos utilizados por la aplicación.

10.2. Visión general de la pipeline

La pipeline está organizada en cuatro pasos encadenados. El primero genera el schema crudo del proveedor a partir de Terraform. El segundo descompone ese schema, que es monolítico y de gran tamaño, en una colección de ficheros individuales, uno por cada recurso o fuente de datos. El tercero produce a partir de cada schema individual una plantilla de código HCL pre-rellena con los atributos del recurso, sus tipos, sus restricciones y sus valores por defecto. El cuarto paso realiza una verificación de la cobertura de tipos y la consistencia de la transformación entre el esquema JSON y la plantilla HCL. El objetivo es garantizar que no se pierde información durante la traducción de JSON a HCL y que todas las estructuras del esquema original están correctamente reflejadas en el artefacto final.

Cada uno de estos pasos está implementado como un programa independiente escrito en Go. La elección de Go se debe a dos razones concretas. Primero, existe una biblioteca oficial mantenida por HashiCorp para interpretar el formato del schema. Segundo, Go es particularmente adecuado para herramientas de procesamiento masivo de ficheros JSON, que es exactamente la naturaleza del problema.

10.3. Paso 1: obtención del schema crudo

El punto de partida es la propia herramienta Terraform. Terraform permite invocar un comando que serializa todo lo que ese binario sabe sobre sí mismo en un único fichero JSON. El proceso no requiere conexión con el proveedor cloud ni credenciales de ningún tipo: el schema describe *qué recursos y que atributos tienen*.

El fichero resultante presenta una estructura jerárquica con tres niveles. En el nivel raíz figura el conjunto de proveedores cargados, identificados por su nombre canónico. Dentro de cada proveedor aparecen cinco categorías de entidades: la configuración del propio proveedor, los **recursos** (entidades que el usuario crea), las **fuentes de datos** (entidades que el usuario consulta sin crearlas), los recursos efímeros y las funciones. Dentro de cada entidad se encuentra la descripción detallada en forma de bloques recursivos: cada bloque puede contener atributos primitivos y sub-bloques, que a su vez contienen sus propios atributos y sub-bloques.

Para AWS en su versión 5.x, este paso produce un fichero monolítico de aproximadamente **trece megabytes** que contiene la descripción completa del proveedor.

10.4. Paso 2: descomposición en schemas individuales

Trabajar directamente sobre el fichero monolítico no es una opción viable. La solución adoptada es descomponer el fichero monolítico en una colección de ficheros individuales, uno por cada recurso o fuente de datos del proveedor, organizados en una jerarquía de directorios que refleja la categorización original. La aplicación consulta entonces el fichero correspondiente al recurso que necesita, sin recorrer el resto. El coste de la descomposición se amortiza una sola vez por versión del proveedor, mientras que el beneficio se materializa en cada consulta posterior.

La decisión técnica más relevante de este paso es que los ficheros se generan **preservando byte a byte** la información original, sin reinterpretarla ni reformatearla semánticamente. Si el proveedor introdujera en una versión futura un campo nuevo en la descripción de los atributos, ese campo quedaría preservado automáticamente, sin que el extractor tenga siquiera que conocer su existencia.

El resultado de este paso, ejecutado sobre el provider de AWS, es una jerarquía de **2.144 ficheros JSON** organizados por categoría: 1.526 recursos, 608 fuentes de datos, 7 recursos efímeros, 3 funciones y la configuración del propio proveedor. Cada fichero conserva exactamente lo que el binario exportó originalmente.

Conviene distinguir aquí entre dos conceptos diferentes que conviven en el proyecto: la **capacidad de la pipeline** y el **catálogo del MVP**. La pipeline es capaz de extraer el catálogo completo del proveedor las 2.144 entidades, y de hacerlo de forma reproducible para cualquier versión futura. El catálogo que la aplicación distribuye y muestra al usuario, en cambio, se ha acotado deliberadamente a un **subconjunto representativo** de recursos AWS, seleccionado por cubrir los escenarios habituales de aprendizaje y prototipado descritos en el alcance del MVP. Esta separación permite validar el flujo completo de la herramienta sobre un catálogo manejable, sin que ello impida ampliar la cobertura simplemente añadiendo más artefactos extraídos por la pipeline, sin tocar el núcleo de la aplicación.

10.5. Paso 3: generación de plantillas HCL

El tercer paso es el más rico desde el punto de vista de diseño. Su objetivo es dado el schema JSON de un recurso, producir un fichero HCL pre-rellenado que liste todos sus atributos y bloques anidados, con valores por defecto sintácticamente válidos y comentarios que documenten tipo, restricciones semánticas y cardinalidad.

La utilidad de estas plantillas en el sistema es doble. Por un lado, son la base sobre la que la

aplicación construye el código IaC real cuando el usuario añade un recurso a su diseño: la plantilla aporta el esqueleto sintáctico y la aplicación rellena los valores reales que el usuario introduce a través de la interfaz visual. Por otro lado, las plantillas son consultables como **material de referencia**: un usuario que se encuentra con un recurso desconocido puede inspeccionar su plantilla para entender de un vistazo qué atributos admite y con qué tipos, sin necesidad de consultar documentación externa.

Para realizar esta traducción es imprescindible interpretar el sistema de tipos que utiliza Terraform internamente. A diferencia de un sistema plano, este sistema admite tipos primitivos (cadenas, números, booleanos), tipos colección (listas, conjuntos y mapas), tipos estructurados (objetos con campos nombrados y tuplas con posiciones) y combinaciones recursivas arbitrarias entre todos los anteriores.

Cada atributo de la plantilla se acompaña, además del comentario de tipo, de un conjunto de **flags semánticos** que indican cómo debe tratarlo el usuario. Campos marcados como *required* es un campo que el usuario debe proporcionar obligatoriamente; aparece en la plantilla con un valor por defecto vacío que el usuario debe sustituir. Un atributo *optional* es un campo que el usuario puede proporcionar o no; aparece de forma similar pero con la anotación correspondiente. La combinación *optional + computed* indica que el usuario puede proporcionar el valor o, si no lo hace, el proveedor lo calculará automáticamente. La plantilla los presenta comentados, de modo que el lector ve que existen y conoce su tipo, pero no afecta a la plantilla.

El generador también traduce a la plantilla la información de **cardinalidad** de los bloques anidados. Cuando el schema indica que un bloque debe aparecer un número mínimo o máximo de veces, ese requisito queda reflejado en un comentario inline junto a la cabecera del bloque, con notaciones del estilo [0..1], [1..*] o [2..5].

10.6. Paso 4: auditoría de cobertura

El cuarto y último paso de la pipeline no produce artefactos consumibles, sino que verifica la corrección del trabajo realizado por los anteriores. Si en una actualización futura del proveedor apareciera un tipo nuevo no contemplado, el generador podría procesarlo de forma genérica.

El auditor recorre el conjunto completo de schemas extraídos y clasifica cada tipo encontrado contra la lista de tipos conocidos. Cualquier tipo que no encaje queda reportado con la ruta exacta donde apareció (recurso y atributo), de modo que la adaptación de la pipeline a la nueva versión del proveedor pueda hacerse de forma atómica.

10.7. Decisiones de diseño relevantes

El diseño de la pipeline tal como se ha descrito es el resultado de varias iteraciones, y algunas decisiones merecen un comentario explícito porque sus razones no son obvias y han condicionado el resto del proyecto.

La primera decisión fue **separar la pipeline en cuatro programas independientes** en lugar de unificarlo en uno solo. Aunque la opción unificada habría sido más simple a primera vista, mantener los pasos separados aporta tres beneficios concretos: cada programa tiene *dependencias mínimas* adaptadas a su tarea, los pasos pueden *ejecutarse de forma independiente* con frecuencias distintas (la extracción solo ante cambios de versión del proveedor; la generación de plantillas y la auditoría cuando convenga), y un fallo en uno de los pasos no compromete a los demás, lo que mejora la robustez global del sistema.

La segunda decisión fue **ejecutar la pipeline una sola vez por versión del proveedor** y no integrarlo en el flujo de ejecución habitual de la aplicación. La generación del catálogo es una operación de *preproceso offline* que produce ficheros estáticos, los cuales pasan a formar parte de la propia distribución de la aplicación. Esto significa que el usuario final no tiene que ejecutar Go ni la pipeline para utilizar la herramienta.

10.8. Síntesis

La pipeline descrita en esta sección resuelve, de forma compacta, cuatro problemas a la vez. Proporciona una representación única, fiable y completa del proveedor cloud sin depender de su documentación pública. Indexa esa representación por nombre de recurso, lo que permite a la aplicación consultar cualquier entidad en tiempo prácticamente constante. Genera material legible para humanos, en forma de plantillas HCL anotadas, que sirve como referencia de cara al usuario. Y verifica, mediante una auditoría automática, que el conjunto de tipos está completamente cubierto, alertando si una evolución futura del proveedor introdujera elementos no contemplados.

El resultado de este pipeline es un **catálogo de recursos** que la aplicación incorpora como parte de su distribución y que constituye la base de todo lo que se describe en las secciones siguientes. Si en el futuro se quisiera incorporar soporte para Google Cloud o Microsoft Azure, bastaría con ejecutar el mismo pipeline contra los proveedores correspondientes y añadir los catálogos resultantes a la distribución, sin necesidad de modificar la aplicación. Esta capacidad de extensión sin cambios en el núcleo es la materialización concreta del subobjetivo de *arquitectura modular y extensible* planteado en la ??.

11. El modelo de dominio

Esta sección se organiza en dos partes complementarias. La primera presenta el **modelo conceptual**: las entidades canónicas, sus relaciones, las reglas de consistencia y el ciclo de vida, formulados con independencia de cualquier tecnología y resumidos visualmente en el diagrama UML de la Figura ???. La segunda parte describe la **implementación** real de ese modelo en el código de la aplicación, donde aparecen las divergencias inevitables entre el diseño ideal y la materialización pragmática, y se acompaña del diagrama UML de la Figura ???.

11.1. Funcionamiento del modelo de dominio

El modelo de dominio representa el estado real del sistema y actúa como **fuentes de verdad**. Su objetivo es capturar la semántica de la infraestructura diseñada por el usuario de forma **neutral** respecto a la interfaz visual y al formato IaC final. Esta separación permite que la UI evolucione sin alterar la lógica y que la generación de IaC cambie sin reescribir el núcleo.

A nivel de diseño, el modelo se construye para cumplir cuatro propiedades: **coherencia** (un único estado válido en cada instante), **trazabilidad** (cada cambio puede seguirse desde la acción del usuario hasta su persistencia), **mantenibilidad** (reglas de negocio concentradas y desacopladas de la UI) y **extensibilidad** (posibilidad de incorporar nuevos recursos o proveedores sin rediseñar el núcleo).

A partir de estas propiedades se define un núcleo estable que abstrae los elementos mínimos del dominio y permite razonar sobre el sistema de forma consistente.

11.2. Núcleo canónico del dominio

La abstracción se organiza en cinco conceptos estables que actúan como vocabulario común para todo el sistema:

- **Proyecto**: contenedor global de trabajo y configuración.
- **Vista**: escenario aislado dentro del proyecto (por ejemplo, un entorno).
- **Recurso**: entidad de infraestructura con identidad estable y tipo lógico.
- **Atributo**: propiedad tipada de un recurso.

- **Relación (enlace):** vínculo semántico entre atributos de recursos.

Este patrón ($\text{Proyecto} \rightarrow \text{Vista} \rightarrow \text{Recurso} \rightarrow \text{Atributo}$, con Enlaces entre atributos de recursos) es el que utiliza el modelo para describir cualquier arquitectura, independientemente de su complejidad o del proveedor cloud al que se destine. La Figura ?? formaliza esta estructura como un **diagrama UML de clases conceptual**, donde se hacen explícitas las **multiplicidades** entre entidades. Más adelante, en la Sección ??, se contrasta este modelo conceptual con su materialización real en el código.

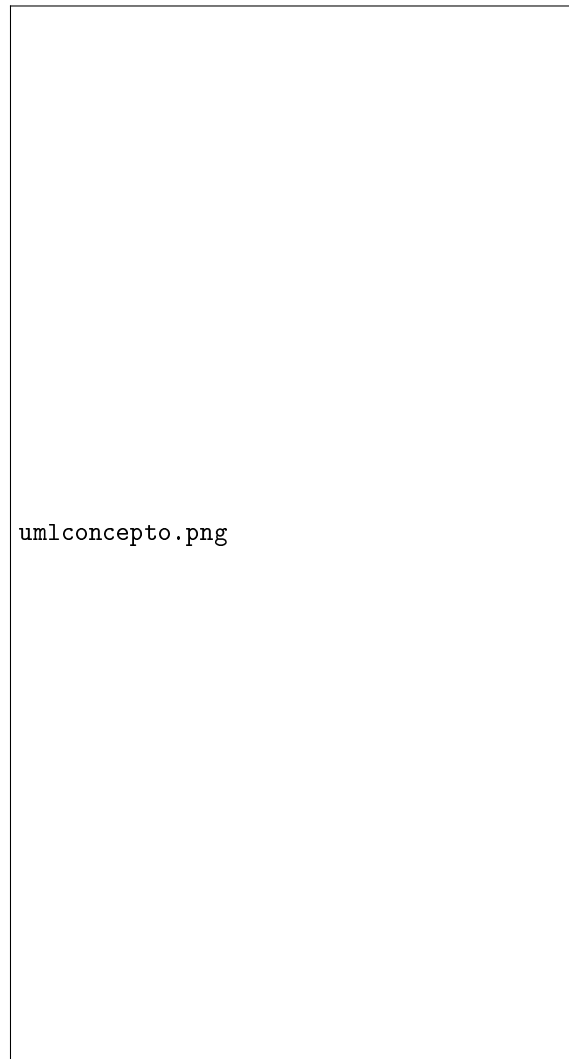


Figura 5: Diagrama UML de clases del modelo de dominio (vista conceptual)
Fuente: elaboración propia

Cada recurso del diseño se representa como una instancia de la entidad *Recurso*, que captura la información mínima necesaria para reconstruir el bloque equivalente en el código IaC y para identificar el recurso unívocamente dentro del proyecto.

En esa estructura, los **atributos** son la representación estructurada de los valores que el usuario ha asignado a las propiedades del recurso. Internamente se almacenan en un mapa plano de claves a valores, donde las claves pueden utilizar un convenio de *rutras compuestas* para representar bloques anidados sin recurrir a estructuras heterogéneas.

La noción de vista introduce un aislamiento operativo. Esta estructura por vistas tiene implicaciones directas en la forma en que el sistema persiste el trabajo del usuario. Una propiedad importante del esquema de persistencia descrito es que cada vista dispone de un directorio de trabajo completamente independiente. Este aislamiento es lo que permite que un mismo proyecto pueda contener simultáneamente una vista de desarrollo desplegada sobre una cuenta cloud y una vista de producción desplegada sobre otra distinta, sin riesgo de mezclas. Es también lo que permite que el usuario abra varias vistas a la vez.

En conjunto, estos conceptos constituyen el metamodelo interno. No dependen del proveedor cloud ni del lenguaje IaC, y son suficientes para describir una arquitectura cloud de manera consistente.

11.2.1. Relación entre dominio, presentación y capa de aplicación

Definido el metamodelo, se concreta cómo interactúan las capas. **La capa de presentación** no modifica directamente el modelo. Su función es capturar eventos de usuario y enviarlos como comandos. **La capa de aplicación** recibe esos comandos y ejecuta la lógica de dominio correspondiente.

El flujo general es el siguiente:

1. La interfaz registra una acción del usuario (crear, editar, conectar o eliminar).
2. La capa de aplicación traduce esa acción a una operación de dominio.
3. El modelo valida invariantes y aplica el cambio si es consistente.
4. El estado actualizado se propaga a la interfaz para su renderizado.
5. El nuevo estado se persiste y se usa para regenerar IaC cuando procede.

Esta arquitectura desacoplada evita que la lógica de negocio quede distribuida entre componentes visuales y facilita la evolución independiente de cada capa.

11.2.2. Reglas de consistencia e invariantes

Para mantener ese metamodelo coherente en el tiempo, el modelo impone reglas de consistencia que se validan en cada operación:

- **Unicidad de identidad:** cada recurso mantiene un identificador único y estable durante toda su vida.
- **Unicidad nominal por vista:** no se permiten colisiones de nombres lógicos entre recursos del mismo tipo dentro de la misma vista.
- **Integridad referencial:** un enlace solo es válido si origen y destino existen y sus atributos son compatibles.
- **No ambigüedad estructural:** un recurso no puede quedar simultáneamente en estados jerárquicos incompatibles.
- **Persistencia coherente:** toda mutación válida del dominio debe poder serializarse sin pérdida semántica.

11.2.3. Ciclo de vida de los elementos del dominio

Estas reglas se aplican durante todo el ciclo de vida de cada entidad, que se modela de forma explícita:

1. **Creación:** al añadir un recurso, se genera su identidad, se inicializan atributos y se registra su pertenencia a la vista activa.
2. **Actualización:** cambios de atributos, conexiones o pertenencias modifican el estado conservando invariantes.
3. **Sincronización:** tras cada cambio, se actualizan las proyecciones visuales y la representación textual.
4. **Eliminación:** al borrar un recurso, se eliminan también referencias y enlaces dependientes para evitar estado huérfano.

Este ciclo de vida asegura que el sistema no entre en estados parciales o contradictorios tras operaciones encadenadas.

11.3. Implementación del modelo

Las subsecciones anteriores describen el **modelo conceptual**: las cinco entidades canónicas (Proyecto, Vista, Recurso, Atributo y Enlace), sus invariantes y su ciclo de vida, formuladas con independencia de cualquier tecnología y resumidas en el diagrama de la Figura ???. Esta subsección describe cómo se materializan esos conceptos en el sistema real; el contraste entre ambos diagramas (Figuras ??? y ???) hace visibles las divergencias inevitables entre el diseño ideal y su implementación.

La implementación se ha realizado íntegramente en **TypeScript** dentro de la capa de aplicación. La capa de infraestructura (Rust/Tauri) no replica el modelo: actúa exclusivamente como puente hacia el sistema de ficheros y la *Terraform CLI*, y recibe el estado ya serializado. Esta decisión concentra toda la lógica de dominio en un único lugar y evita el coste de mantener dos representaciones sincronizadas a ambos lados del puente Tauri.

En el código, las cinco entidades conceptuales se traducen en los tipos que recoge la Figura ???. El **Proyecto** (`Project`) actúa como contenedor de versión, metadatos, configuración y la lista de vistas. La **Vista** (`ViewSnapshot`) agrupa los recursos, los nodos visuales y las aristas pertenecientes a un escenario. El **Recurso** (`TerraformResource`) encapsula tanto la información semántica (tipo lógico, identidad, configuración) como la representación visual asociada en el lienzo. El **Enlace** (`SerializedEdge`) modela la conexión visual entre dos nodos del lienzo y, en su interior, contiene una lista de *enlaces* de atributos (`CanvasEdgeMapping`) que son los que realmente realizan el vínculo semántico entre propiedades de recursos. Por su parte, el **Atributo** no aparece como una entidad independiente con su propia clase: se materializa como entradas de un diccionario plano dentro de la configuración del recurso (`ResourceConfig.attributes`), donde la clave admite el convenio de rutas compuestas comentado anteriormente.

Comparando ambos diagramas (Figuras ??? y ???) emergen dos divergencias relevantes respecto al modelo conceptual. En primer lugar, la separación dominio/presentación no se traduce en clases físicamente disjuntas: la información visual (posición, icono, geometría de las aristas) se aloja junto a la información semántica dentro de los mismos tipos. La separación se mantiene a nivel **lógico**: las reglas de dominio operan únicamente sobre los campos semánticos y la capa de presentación es la única que lee y escribe los campos visuales. En segundo lugar, el Enlace tiene una estructura de **dos niveles**: un enlace visual (`SerializedEdge`) entre dos nodos puede contener varios *enlaces* de atributos, lo que permite expresar con una única arista visual múltiples referencias HCL entre los recursos conectados.

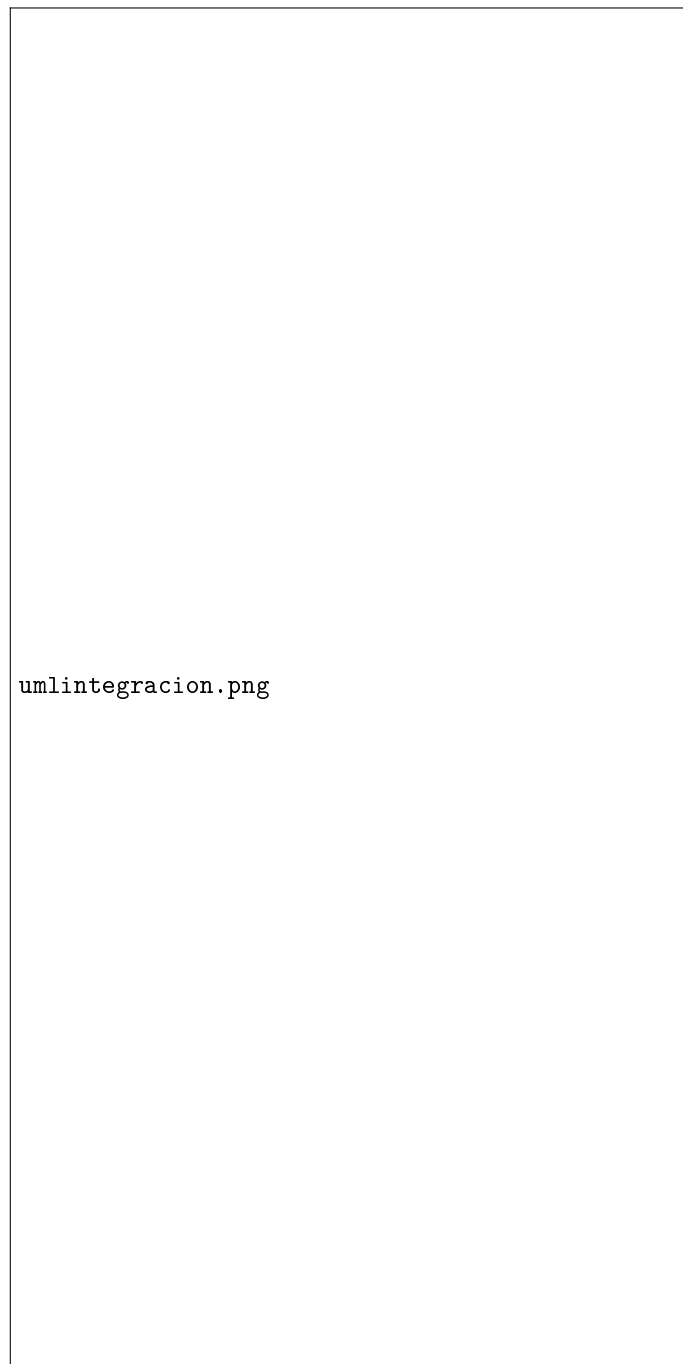


Figura 6: Diagrama UML de clases del modelo de dominio (vista de implementación)
Fuente: elaboración propia

11.4. Persistencia y formato de proyecto

Una vez definido el comportamiento del modelo, se detalla cómo se almacena y recupera el estado del proyecto. La persistencia del modelo de datos en el sistema de ficheros del usuario combina dos formatos complementarios. Por un lado, el modelo completo del proyecto se serializa en un **fichero descriptor con extensión propia**, que contiene en formato JSON toda la información estructurada: metadatos, configuración, vistas, recursos, nodos y aristas. Este fichero es la unidad que el usuario abre, guarda y comparte. Su formato JSON es legible, diff-able con herramientas estándar y susceptible de versionarse con Git como cualquier otro artefacto de un proyecto de software, lo que cumple el requisito de **portabilidad**.

Mantener esta información dentro del modelo de la vista, en lugar de derivarla cada vez del modelo de dominio o del código IaC, es lo que permite que la experiencia del usuario sea *idempotente*.

Por otro lado, junto al fichero descriptor se mantiene una **carpeta hermana** cuya estructura interna espeja la organización lógica del proyecto: dentro de ella, cada vista dispone de su propio subdirectorío, que contiene el fichero de configuración IaC generado a partir del modelo de la vista y cualquier fichero auxiliar que el usuario haya añadido (como variables, locales o módulos). Esta carpeta hermana es precisamente el directorio de trabajo sobre el que opera la herramienta IaC.

Desde el punto de vista de trazabilidad, cada guardado representa un *snapshot* consistente del proyecto. Esto facilita auditoría de cambios, integración con control de versiones y recuperación de sesiones de trabajo sin pérdida de contexto.

11.5. Síntesis

El conjunto compone un modelo explícito, serializable y portable que constituye la fuente de verdad del trabajo del usuario, sobre la que se proyectan todas las representaciones que la aplicación ofrece.

En síntesis, el modelo de dominio no solo organiza datos: define el contrato lógico del sistema. La presentación, la generación de IaC y la integración con herramientas externas dependen de él, pero no lo sustituyen. Esta decisión arquitectónica permite que el proyecto evolucione de forma controlada y verificable, manteniendo la coherencia interna del sistema en todo momento.

12. Capa de presentación

Tras definir el modelo de dominio en la Sección ?? y el catálogo de recursos que lo nutre en la Sección ??, esta sección describe la **capa de presentación**, el contenedor situado en la parte superior del diagrama de contenedores de la Figura ?. Su papel dentro de la arquitectura es doble: por un lado, actúa como **proyección visual** del estado del modelo de dominio, traducéndolo a los elementos gráficos que el usuario percibe en la interfaz; por otro, funciona como **punto de entrada de las acciones del usuario**, capturando sus eventos y reenviándolos como comandos hacia la capa de aplicación, que es quien realmente modifica el modelo. En coherencia con la separación de responsabilidades descrita en la Sección ??, en este nivel **no se implementa lógica de negocio**: la presentación ni decide la validez de una operación, ni genera código IaC, ni se comunica con Terraform.

12.1. Estructura general de la interfaz

La interfaz se organiza en torno a una barra superior y cuatro paneles principales que conforman el espacio de trabajo del usuario.

- **Barra superior:** permite cambiar entre vistas del proyecto y aloja los accesos a las acciones principales. En ella se sitúan los botones de **plan**, **apply** y **destroy**, que actúan como accesos directos a las operaciones disponibles sobre la vista activa.
- **Panel izquierdo:** catálogo de recursos arrastrables, organizado por proveedor y preparado para búsqueda y filtrado.
- **Lienzo central:** zona principal de edición visual, donde se construye la arquitectura mediante arrastrar y soltar.
- **Panel derecho:** inspector de propiedades del recurso seleccionado, con los campos editables del modelo.
- **Panel inferior:** zona reservada a la terminal embebida, al mapa de objetos y a los logs y mensajes de validación.

Esta disposición responde a un patrón habitual en herramientas de diseño visual: el usuario dispone de una zona de catálogo, una zona de edición, una zona de propiedades y una zona de retroalimentación. Además, la distribución de paneles permite adaptar la interfaz a diferentes flujos de trabajo sin alterar la lógica subyacente.

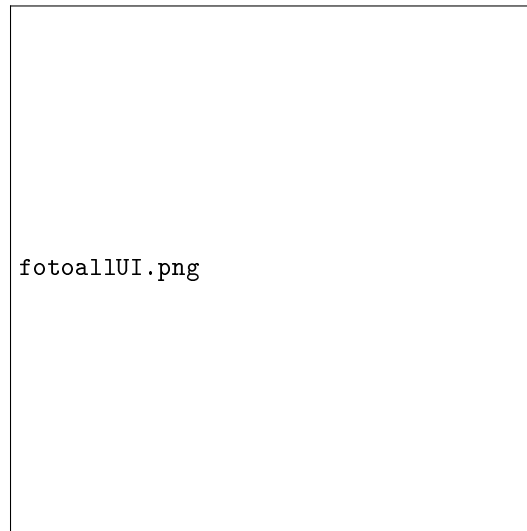


Figura 7: Vista general de la interfaz con los cinco paneles anotados
Fuente: elaboración propia

12.2. Catálogo de recursos

El panel izquierdo presenta el catálogo de recursos extraído por la pipeline descrito en la Sección ???. Cada recurso se muestra como un elemento arrastrable con información suficiente para reconocerlo de forma rápida: icono, nombre legible y nombre técnico de Terraform.

La interfaz incorpora mecanismos de búsqueda y filtrado para reducir la complejidad de un catálogo potencialmente amplio. El objetivo es que el usuario encuentre con facilidad el recurso deseado sin necesidad de navegar manualmente por toda la lista. Desde el punto de vista de la experiencia de uso, este panel actúa como punto de entrada al diseño de la arquitectura.



Figura 8: panel izquierdo: catálogo de recursos con búsqueda activa
Fuente: elaboración propia

12.3. Inspector de propiedades

Cuando un recurso está seleccionado en el lienzo, el panel derecho muestra un **inspector** que lista todos los atributos del recurso con sus valores actuales. El inspector es la vía por la cual el usuario edita las propiedades sin tocar código.

Para cada atributo, el inspector elige automáticamente el editor más adecuado según su tipo:

- **String/number/bool**: input, textarea, toggle.
- **list(string)**: lista de inputs con add/remove.
- **object/set(object(...))**: bloque expandible con sub-inputs por campo.
- **computed (solo lectura)**: input deshabilitado, con tooltip “Terraform calcula este valor”.

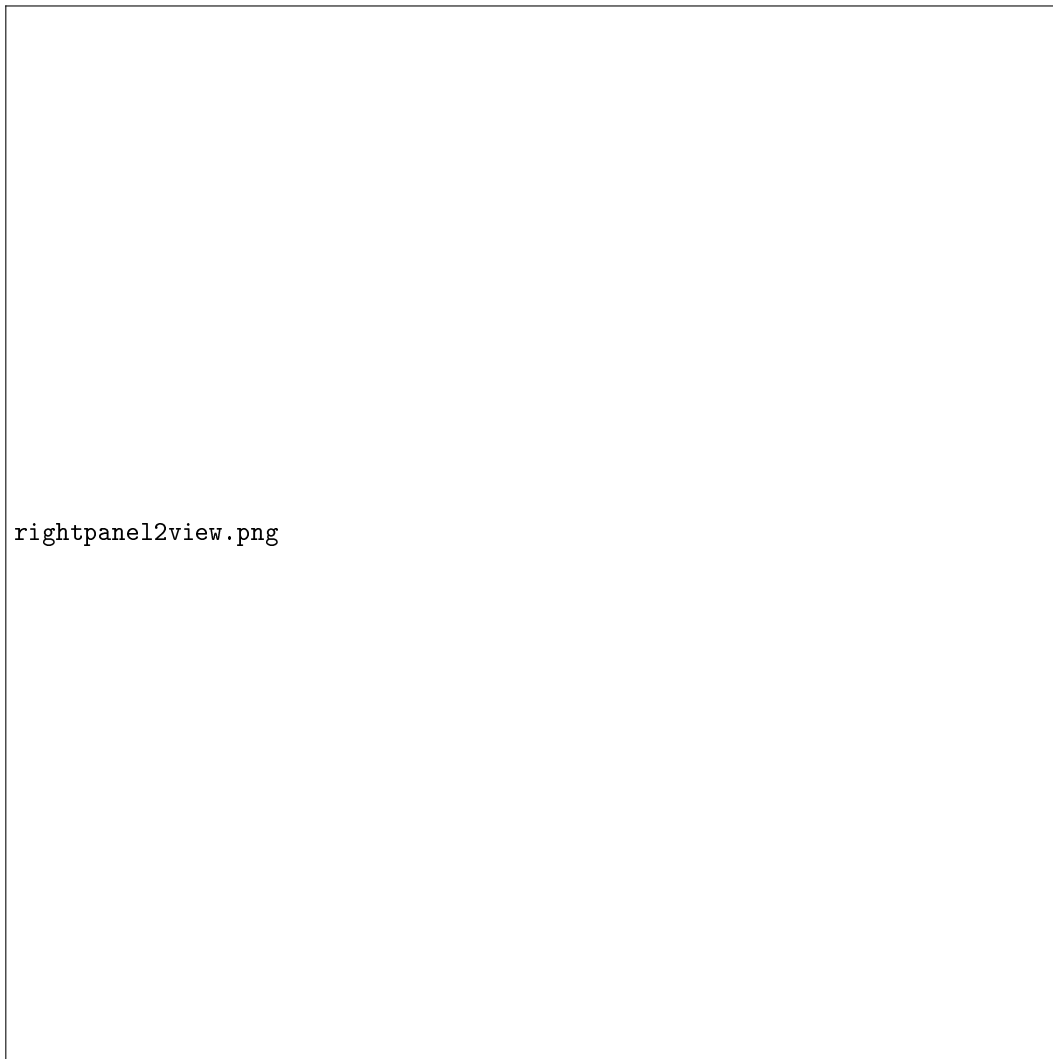


Figura 9: panel derecho: inspector mostrando distintos tipos de editor
Fuente: elaboración propia

12.4. Panel inferior: terminal y logs

El panel inferior aloja tres responsabilidades:

- **Terminal embebida:** Una PTY real que ejecuta comandos en el directorio de la vista actual. El usuario puede invocar `terraform plan`, `terraform apply`, u otros comandos, y ver

la salida en tiempo real. La terminal puede además **desacoplarse en una ventana independiente**, de modo que el usuario pueda seguir trabajando sobre el lienzo principal mientras observa la ejecución de Terraform en paralelo, sin perder espacio del panel inferior.

- **Mapa de objetos:** Un visor que representa, de forma jerárquica y estructurada, los recursos actuales de la vista con sus atributos y relaciones. Cumple un doble propósito: permitir al usuario inspeccionar de un vistazo el estado del modelo sin tener que recorrer el lienzo recurso por recurso, y servir como vista de depuración cuando algo no cuadra entre el diseño visual y el HCL generado. Se actualiza automáticamente con cada cambio del modelo de dominio.
- **Logs y validación:** Eventos de validación del código HCL, errores de referencia, warnings. Se alimentan desde la capa de aplicación cada vez que el usuario modifica el código.

Estos elementos proporcionan retroalimentación inmediata.



Figura 10: panel inferior: terminal embebida, object mapper y logs
Fuente: elaboración propia

12.5. Editor de código HCL

Además del canvas, la interfaz incorpora una vista de código que permite visualizar la definición **HCL** generada a partir del diseño. Esta vista resulta útil para usuarios que desean revisar el resultado textual de la arquitectura sin abandonar el entorno visual.

El editor de HCL forma parte de la capa de presentación porque solo muestra y permite editar el contenido textual asociado a la vista activa. La interpretación semántica del código, su validación y su sincronización con el modelo de dominio corresponden con la capa de aplicación. Desde la experiencia de usuario, esta vista actúa como complemento del lienzo y facilita una transición gradual entre modelado visual y configuración textual.

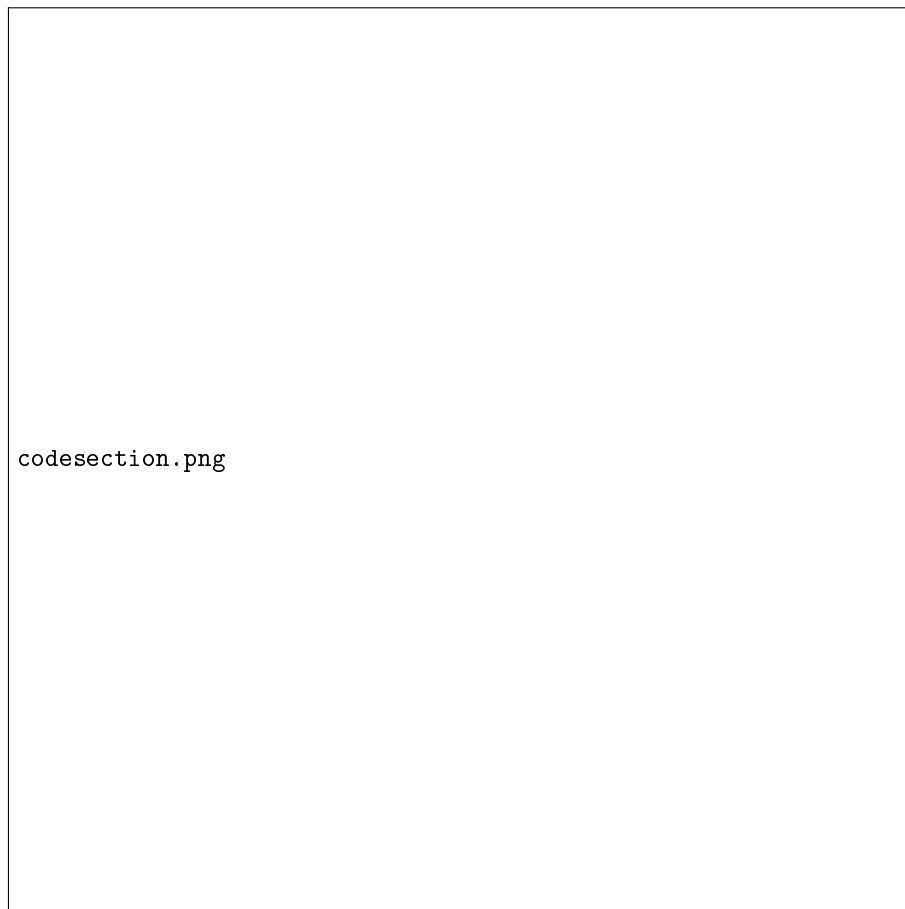


Figura 11: editor de código HCL con un main.tf de ejemplo
Fuente: elaboración propia

12.6. Vistas Diff y Cloud

La interfaz incluye dos proyecciones adicionales del estado del sistema: **Diff** y **Cloud**. Ambas se entienden como distintas proyecciones del mismo núcleo de información, no como modelos independientes. La capa de presentación se limita a mostrarlas de forma clara y consistente para que el usuario pueda analizar el estado de la infraestructura desde perspectivas complementarias.

Vista Diff

La vista Diff (Figura ??) representa los cambios previstos a partir de la comparación entre el estado actual del proyecto y el resultado de una operación de planificación. Esta visualización permite identificar de forma rápida qué recursos se crearán, modificarán o eliminarán.

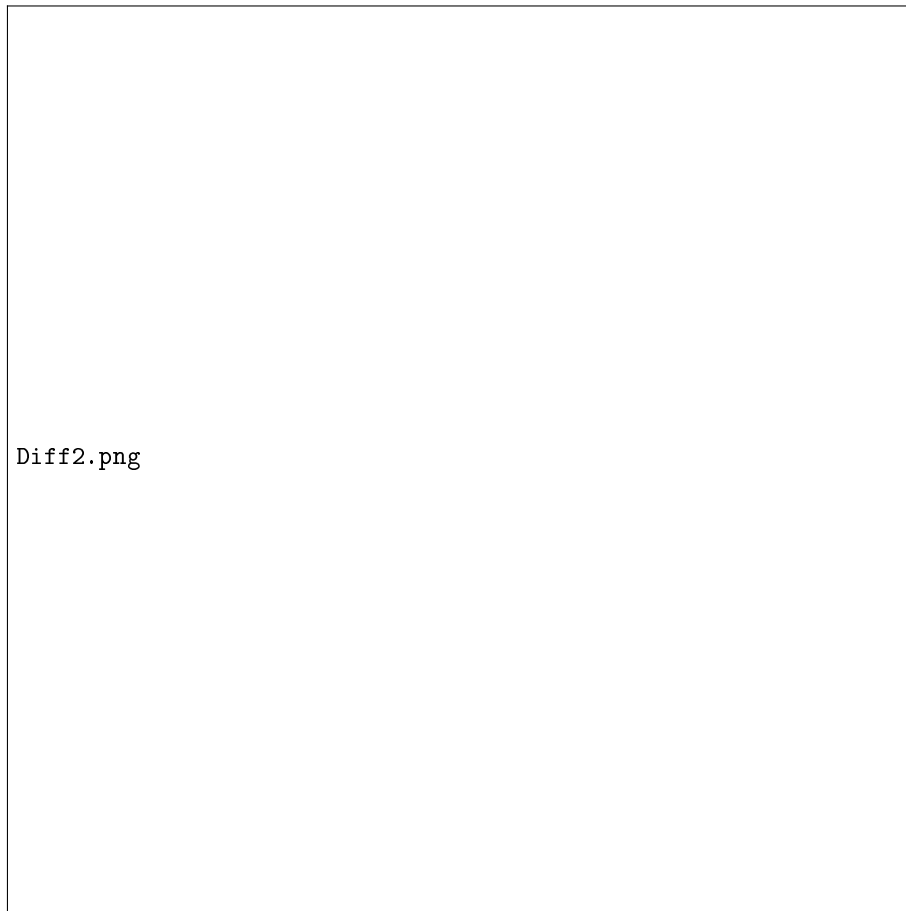


Figura 12: vista Diff: cambios previstos por el último plan
Fuente: elaboración propia

Vista Cloud

La vista Cloud (Figura ??), por su parte, muestra el estado real recuperado del proveedor después de consultar la infraestructura desplegada. Su función es ofrecer una referencia visual del entorno existente y facilitar la comparación entre lo diseñado, lo generado y lo realmente desplegado.

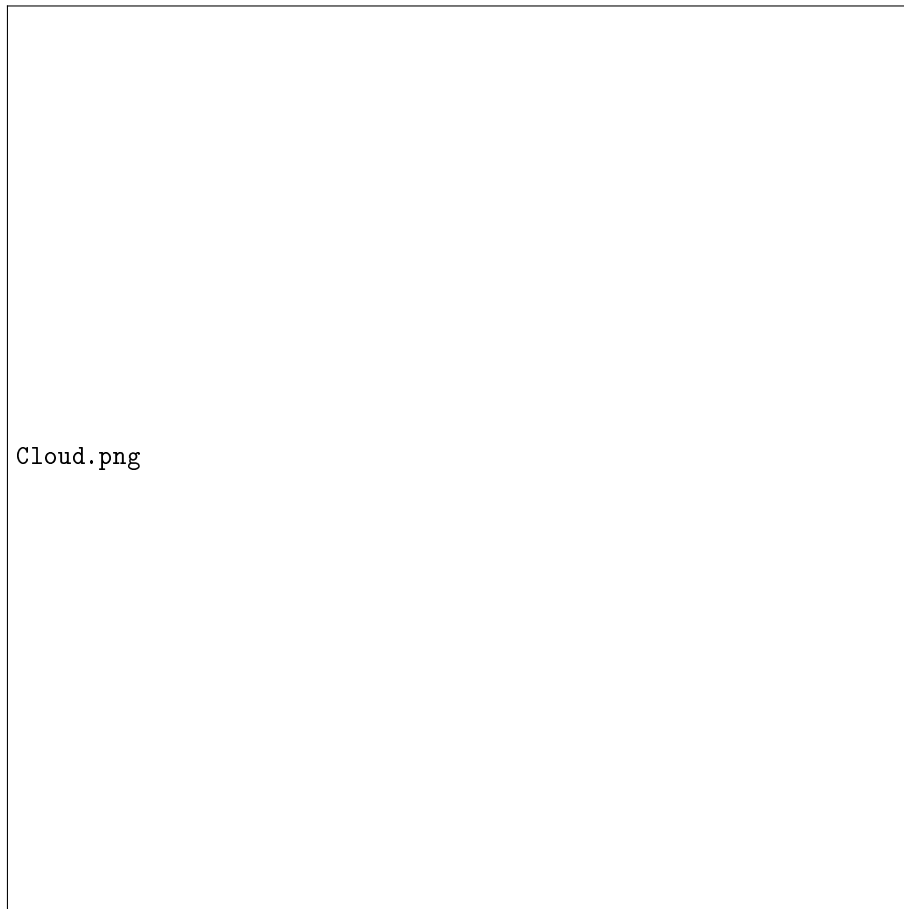


Figura 13: vista Cloud: estado real recuperado del proveedor
Fuente: elaboración propia

12.7. Relaciones entre recursos: aristas y enlaces

Los recursos cloud intercambian datos entre sí mediante referencias. En la presentación, esto se visualiza como **aristas** (líneas) que conectan dos recursos. Cada arista no es decorativa: es un contenedor de información que declara uno o más **enlaces**.

Un enlace especifica con precisión qué atributo del recurso origen alimenta qué atributo del recurso destino. Por ejemplo, una arista entre una VPC y una subnet puede contener un enlace que indica que el atributo `id` de la VPC alimenta al atributo `vpc_id` de la subnet. El usuario crea la arista visualmente y el enlace queda registrado en el modelo, traducándose en la referencia correspondiente

dentro del HCL.

Cuando el usuario crea un enlace, ocurre una operación doble:

1. El enlace se registra en la arista, haciéndose visible como una entrada en la UI.
2. El modelo de dominio se actualiza: el atributo correspondiente del recurso destino se rellena con la expresión que apunta al recurso origen.

Si el usuario elimina un enlace, la arista se limpia y el atributo del recurso destino se vacía, manteniendo coherencia entre lo visual y lo estructural.

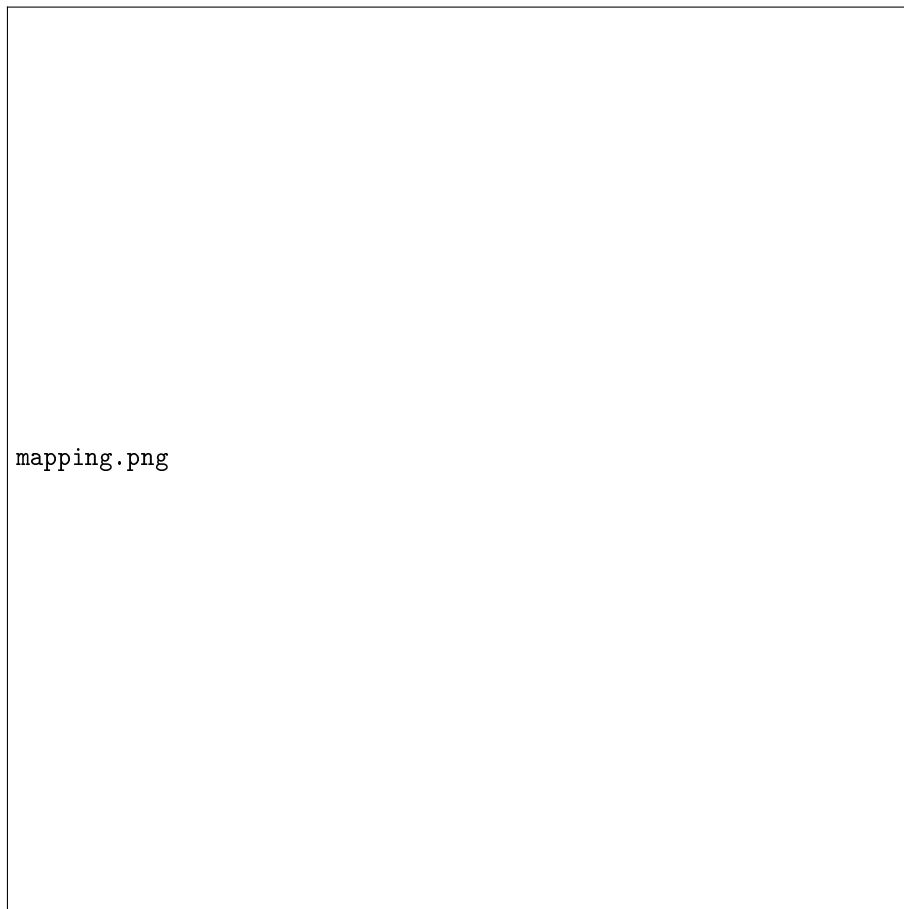


Figura 14: vista Enlace: arista con un enlace configurado entre dos recursos
Fuente: elaboración propia

12.8. Síntesis

En conjunto, la capa de presentación actúa como un *espejo* del modelo de dominio. No toma decisiones sobre la validez de operaciones, no genera código, no habla con Terraform. Su única responsabilidad es:

1. **Observar** cambios en el modelo de dominio y re-renderizar.
2. **Capturar** eventos del usuario y enviarlos a la capa de aplicación.
3. **Visualizar** la estructura del proyecto de forma intuitiva: recursos, contenedores, aristas, atributos.

Esta separación clara permite que la presentación evolucione (cambiar colores, layouts, bibliotecas de UI) sin afectar la lógica del dominio. Inversamente, cambios en el modelo de dominio no rompen la presentación siempre que la interfaz se mantenga.

13. Capa de aplicación

La capa de aplicación es el núcleo lógico del sistema. Su función es implementar la lógica de negocio, procesar los comandos recibidos desde la presentación, actualizar el modelo de dominio y coordinar la generación de código IaC. Es el puente entre lo visual y lo textual, entre la intención del usuario y la realidad de la infraestructura.

13.1. Responsabilidades principales

La capa de aplicación asume un conjunto de responsabilidades bien definidas:

- **Gestión del modelo de dominio:** crea, actualiza y elimina recursos, atributos y relaciones (aristas/enlaces) de forma consistente.
- **Sincronización modelo–HCL:** convierte el modelo en configuración Terraform válida y, cuando el usuario edita el HCL, reconcilia esos cambios con el modelo sin romper su estructura.
- **Persistencia del proyecto:** serializa el estado completo en el fichero descriptor y mantiene la carpeta de trabajo de cada vista con su `main.tf` y ficheros auxiliares.
- **Aplicación de reglas semánticas:** evita incoherencias como atributos inválidos, referencias rotas o estados intermedios que el usuario no debería observar.
- **Coordinación con Terraform:** prepara la ejecución (sync de ficheros, credenciales, contexto de la vista) y procesa resultados (plan, validación, estado cloud) para reflejarlos en la interfaz.
- **Historial de operaciones:** registra los cambios significativos del modelo para permitir al usuario deshacer y rehacer acciones de forma reversible.

13.2. Procesamiento de comandos

Cada acción del usuario se traduce en un comando que la capa de aplicación recibe para su procesamiento. El comando encapsula la intención del usuario en términos de operaciones sobre el modelo de dominio (p. ej., “crear un recurso de tipo `aws_vpc` con nombre `my_vpc`”).

El procesamiento de comandos sigue un patrón general:

1. Validación: se comprueba que el comando es semánticamente correcto y no viola invariantes.

2. Ejecución: se actualiza el modelo de dominio según lo especificado por el comando.
3. Notificación: se informa a la presentación que el modelo ha cambiado para que re-renderice.
4. Persistencia: se guarda el nuevo estado del proyecto en disco.
5. Generación: si procede, se actualiza el código IaC asociado a la vista activa.

Este flujo asegura que cada acción del usuario se refleja de forma consistente en el estado interno, en la interfaz y en el código generado.

Catálogo de comandos representativos

La Tabla ?? recoge los comandos representativos que esta capa procesa. Cada comando se describe por su intención y por su efecto sobre el modelo de dominio.

Comando	Efecto sobre el modelo
<i>Crear recurso</i>	Añade una entidad recurso al modelo con identidad fresca, posición inicial y atributos por defecto tomados del catálogo.
<i>Editar atributo</i>	Actualiza el valor de un atributo del recurso, validando contra el tipo declarado en el catálogo.
<i>Renombrar recurso</i>	Cambia el nombre lógico, verificando la unicidad nominal dentro de la vista activa.
<i>Mover recurso</i>	Actualiza la posición visual del recurso sin alterar su semántica.
<i>Eliminar recurso</i>	Elimina el recurso y, en cascada, todos los enlaces que lo referencian, evitando estado huérfano.
<i>Crear enlace</i>	Registra una nueva arista, valida que origen y destino existen y son compatibles, y actualiza el atributo destino con la referencia correspondiente.
<i>Eliminar enlace</i>	Limpia la arista y revierte el atributo destino al valor por defecto.
<i>Crear vista</i>	Añade una vista al proyecto con su directorio de trabajo aislado.
<i>Cambiar vista activa</i>	Cambia el contexto de la sesión; no muta el modelo, pero reproyecta presentación y código sobre la vista elegida.
<i>Importar HCL</i>	Parsea el código y lo reconcilia con el modelo (sentido HCL → modelo de la sincronización bidireccional).

Cuadro 11: Comandos representativos procesados por la capa de aplicación
Fuente: elaboración propia.

Si en cualquier punto se viola una invariante (por ejemplo, una colisión nominal o un valor incompatible con el tipo), la mutación se aborta antes de ejecutarse y se devuelve un error a la presentación,

dejando el modelo en su estado anterior. Esta atomicidad es la que garantiza que el proyecto nunca quede en un estado parcialmente modificado.

13.3. Sincronización bidireccional entre modelo y código

Un rasgo distintivo de la aplicación es que el modelo visual y el HCL no son dos fuentes de verdad independientes. La capa de aplicación mantiene ambos sincronizados mediante un ciclo bidireccional:

- **Modelo** → **HCL**: cualquier cambio en el modelo (nuevo recurso, edición de atributos, enlace entre recursos) dispara la regeneración del código HCL para la vista activa. La salida es determinista: a mismo modelo, mismo HCL.
- **HCL** → **Modelo**: cuando el usuario edita el código HCL desde la vista de código, el texto se parsea y se reconcilia con el modelo. Se actualizan valores de atributos, pero se preservan elementos que el modelo necesita para seguir siendo válido (por ejemplo, geometría del lienzo o metadatos internos).

Este enfoque evita la divergencia entre lo visual y lo textual, y permite que el usuario alterne entre ambos sin romper la coherencia del proyecto.

13.4. Historial de operaciones

La capa de aplicación mantiene un **historial en memoria** de las operaciones realizadas sobre el modelo durante la sesión activa. Este historial habilita las acciones de *deshacer* y *rehacer*, que el usuario puede invocar directamente desde la interfaz. Cada entrada del historial corresponde a un cambio significativo del modelo creación o eliminación de un recurso, edición de un atributo, alta o baja de un enlace, y se almacena con la información mínima necesaria para revertirlo. Las acciones puramente visuales, como el desplazamiento de un nodo en el lienzo, no se registran como pasos independientes para evitar saturar el historial con cambios cosméticos. referencia.

13.5. Integración con operaciones Terraform

Cuando el usuario solicita *plan*, *apply*, *destroy* o *show*, la capa de aplicación prepara el entorno: sincroniza los ficheros del proyecto, valida que la vista activa está lista y delega la ejecución al backend nativo. Después, interpreta la salida para actualizar el estado visual y el estado real de

la infraestructura. De esta forma, el resultado de Terraform se traduce en feedback directo para el usuario, sin que tenga que abandonar la herramienta.

Es importante notar que la capa de aplicación **no invoca Terraform directamente**: únicamente prepara el contexto y delega la ejecución en la capa de infraestructura, que se ocupa de la mecánica del subproceso, del *streaming* de salida y de la gestión de la confirmación interactiva. La capa de aplicación se mantiene así libre de detalles del sistema operativo y puede razonarse sobre ella de forma aislada, lo que también facilita su verificación.

13.6. Traducción visual a IaC

La traducción visual a IaC es el núcleo operativo de esta capa. A partir del modelo de dominio, se genera un `main.tf` completo que Terraform puede ejecutar directamente. La generación es determinista e idempotente: el mismo modelo produce siempre el mismo HCL, lo que facilita trazabilidad y control de cambios.

Entrada: modelo de dominio y catálogo

La traducción toma como entrada el conjunto de recursos del modelo (tipo, nombre, atributos y relaciones) y el catálogo de schemas generado por la pipeline. El catálogo aporta estructura y tipos para el inspector y el autocompletado, mientras que el modelo aporta los valores concretos y las referencias entre recursos definidas en el lienzo.

Reglas de mapeo principales

La generación sigue reglas deterministas:

- **Recurso → bloque HCL**: cada recurso se convierte en un bloque `resource` (o `data`) con su `type` y `name`.
- **Atributos planos → asignaciones**: las claves simples se serializan como `clave = valor`.
- **Atributos con rutas → sub-bloques**: las claves con notación `a.b.c` se expanden a bloques anidados con la estructura correspondiente.
- **Estructura global**: el fichero resultante incluye un bloque `terraform { required_providers { ... } }` y el bloque `provider 'x' { region = ... }` antes de los recursos.

Enlaces y referencias entre recursos

Las aristas del lienzo representan referencias entre recursos. En el modelo, un enlace se almacena como una expresión de Terraform (por ejemplo `aws_vpc.main.id`). Durante la traducción, ese valor se escribe sin comillas para que Terraform lo interprete como referencia y no como literal.

Contenedores y atributos implícitos

La posición de un recurso dentro de contenedores visuales también se traduce a HCL. Un nodo colocado dentro de un contenedor jerárquico (por ejemplo, una VPC o una subnet) hereda referencias implícitas: se completan atributos como `vpc_id` o `subnet_id` con la dirección del recurso padre. En el caso de contenedores de zona (por ejemplo, *availability zone* o *security group*), la pertenencia se refleja como listas de referencias o valores de zona, respetando la semántica de cada recurso. Esta lógica evita que el usuario tenga que cablear manualmente relaciones estructurales y garantiza que el HCL sea coherente con la organización visual del lienzo.

Bloques repetibles y colecciones

Cuando un atributo es una colección de objetos, se generan bloques repetidos en lugar de una única asignación con lista. Esta decisión se alinea con la sintaxis esperada por Terraform para `list(object(...))` y `set(object(...))`, y permite que cada entrada se exprese como un bloque anidado independiente.

Normalización de valores

Antes de escribir el HCL, se descartan valores vacíos o no significativos para evitar configuraciones inválidas. Los tipos se respetan para producir literales correctos: booleanos y números se escriben sin comillas, las cadenas se serializan con comillas, y las referencias se escriben como expresiones.

Resultado y ejemplo mínimo

El resultado es un `main.tf` autocontenido, legible y coherente con el diseño visual. La generación es idempotente y conservadora: solo se materializa lo que el modelo define, sin introducir ruido ni valores por defecto no solicitados.

Si el usuario coloca un `aws_vpc` y un `aws_subnet`, y crea un enlace desde `aws_vpc.main.id` hacia `subnet.vpc_id`, el HCL generado incluye la referencia de forma explícita:

```
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}
resource "aws_subnet" "public" {
  vpc_id      = aws_vpc.main.id
  cidr_block = "10.0.1.0/24"
}
```

Listing 1: VPC con subnet pública

Fuente: elaboración propia

14. Capa de infraestructura

La capa de infraestructura es el nivel más bajo del sistema y concentra todo lo que depende del sistema operativo: ejecución de binarios, acceso a disco, procesos interactivos y lectura de credenciales locales. Se implementa en el backend nativo en Rust (Tauri) y expone comandos al frontend mediante *invoke*, de forma que la interfaz no tenga acceso directo a recursos sensibles. Esta capa no conoce la lógica de negocio del modelo; su misión es ejecutar acciones fiables y transmitir eventos de vuelta a la aplicación.

14.1. Rol, límites y superficie de comandos

La capa se materializa en un conjunto de comandos Tauri que encapsulan todo el ciclo de Terraform: `plan`, `plan -destroy`, `apply`, `destroy`, `show -json`, validación `validate -json` y diagnósticos mediante LSP. Además, expone comandos auxiliares para confirmar ejecuciones interactivas (envío de `yes/no`), calcular una firma del `terraform.tfstate` y gestionar una terminal embebida. De este modo, la capa de aplicación opera siempre sobre una API controlada y predecible.

14.2. Ejecución de Terraform por vista

Cada vista del proyecto se ejecuta en su propio directorio Terraform. Esto implica que cada comando se lanza con `cwd` apuntando a la carpeta de la vista activa, evitando mezclar estados entre vistas y permitiendo trabajar con entornos distintos (por ejemplo, *dev* y *prod*) dentro del mismo proyecto. Antes de cada comando se asegura la sincronización del `main.tf` y de los ficheros auxiliares en disco.

14.3. Procesos interactivos y confirmación de `apply/destroy`

Los comandos `apply` y `destroy` son interactivos: Terraform solicita confirmación. La capa de infraestructura mantiene el proceso abierto con `stdin` conectado y guarda una referencia segura al descriptor para poder enviar `yes` o `no` cuando el usuario confirma desde la UI. Este diseño evita que el usuario escriba manualmente en una consola y mantiene el flujo dentro de la aplicación.

Además de confirmar, el usuario puede **cancelar** en cualquier momento un proceso en curso. En ese caso, la capa de infraestructura envía al subprocesso una **señal de interrupción** (equivalente a `SIGINT`) y, si tras un breve margen el proceso no ha terminado, escala a una señal de terminación

(SIGTERM). Este mecanismo está abstraído por sistema operativo, de modo que el comportamiento es equivalente en Linux, macOS y Windows. La cancelación cierra la ejecución de forma controlada y devuelve el control a la aplicación sin dejar procesos huérfanos.

14.4. Streaming de salida en tiempo real

La salida estándar y de error se transmite en vivo. Se crean lectores concurrentes para `stdout` y `stderr`, se envían por un canal y se emiten eventos Tauri con cada línea.

14.5. Ejemplo: flujo Plan → Diff → Cloud

Un flujo típico de infraestructura se ejecuta así:

1. El usuario pulsa `plan`. La capa de infraestructura ejecuta `terraform plan` en el directorio de la vista activa y transmite la salida en streaming.
2. La capa de aplicación interpreta el plan y proyecta los cambios en la vista Diff: nodos marcados como *create*, *change* o *destroy*.
3. Tras aplicar los cambios, se consulta `terraform show -json` y se actualiza la vista Cloud con los recursos reales y sus valores resultantes (IDs, ARNs, IPs).

Este ejemplo resume cómo la capa de infraestructura convierte el resultado de Terraform en información accionable para el usuario, manteniendo el ciclo visual conectado con el estado real.

14.6. Gestión de credenciales y aislamiento

La capa resuelve credenciales AWS justo antes de cada ejecución, sin persistirlas en disco. Puede obtenerlas de tres fuentes: variables de entorno, perfiles locales (`~/.aws/credentials`) o un fichero `.env`. Las credenciales se inyectan como variables de entorno del subprocesso (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_SESSION_TOKEN`, `AWS_DEFAULT_REGION`) y se descartan al finalizar la ejecución. Esta estrategia reduce la exposición y evita que la interfaz pueda leer valores sensibles.

14.7. Estado real y detección de cambios

Para el modo Cloud, la capa ejecuta `terraform show -json` y devuelve un listado normalizado de recursos del *state*. La aplicación lo indexa y compara con el modelo para marcar cada nodo como *present* o *missing*. En paralelo, se calcula una firma ligera del `terraform.tfstate` (longitud y *mtime*) para detectar cambios externos sin reparsear el fichero continuamente.

14.8. Terminal embebida y comandos auxiliares

Además de Terraform, la infraestructura expone una PTY real para ejecutar comandos dentro del directorio de la vista. Esto habilita la terminal embebida, la ventana desacoplada y tareas de depuración avanzadas sin abandonar la aplicación, manteniendo el flujo de trabajo en un único lugar.

14.9. Seguridad y portabilidad

El diseño prioriza la separación de responsabilidades: la interfaz no accede directamente al sistema de ficheros ni a los binarios, y todas las operaciones críticas se concentran en el backend nativo. La capa abstrae diferencias de rutas, permisos y ejecución entre sistemas operativos, manteniendo el mismo comportamiento en Linux, macOS y Windows.

15. Conclusiones

15.1. Conclusiones generales del proyecto

Este Trabajo Final de Grado planteó el diseño y desarrollo de un sistema visual para la creación y despliegue de arquitecturas cloud mediante Infraestructura como Código, con un enfoque open source y neutral respecto al proveedor. A lo largo del documento se ha cubierto la justificación del problema, el alcance, la planificación, el diseño arquitectónico, la implementación de las cuatro capas, la pipeline de extracción de recursos y la validación funcional sobre casos representativos.

El resultado entregado es una aplicación de escritorio funcional capaz de modelar visualmente arquitecturas cloud, generar el HCL correspondiente de forma determinista y ejecutar el ciclo completo *plan* $\rightarrow$ *apply* $\rightarrow$ *destroy* sobre un proveedor real, sin que el usuario tenga que abandonar el entorno visual. La aplicación incorpora un catálogo extraído de forma automática a partir del schema oficial de Terraform, lo que garantiza fidelidad y abre la puerta a la incorporación de nuevos proveedores sin modificar el núcleo de la aplicación.

El proyecto, planteado como un MVP, alcanza sus objetivos y deja sentadas las bases sobre las que pueden construirse las evoluciones descritas más adelante.

15.2. Evaluación de la propuesta

Los cuatro subobjetivos definidos en la Sección ?? se han materializado de forma verificable en la solución entregada:

- **Interfaz gráfica para modelar arquitecturas cloud:** se ha logrado una interfaz visual completa, descrita en la Sección ??, que permite al usuario diseñar arquitecturas mediante arrastrar y soltar recursos desde un catálogo, editar sus atributos en un inspector contextual y conectarlos entre sí mediante enlaces. La interfaz integra además un editor HCL embebido, una terminal y vistas Diff y Cloud que proporcionan retroalimentación inmediata sobre el estado del proyecto y de la infraestructura desplegada.
- **Traducción automática a IaC:** se describe en la sección de traducción visual a IaC y se valida con la ejecución de `terraform validate` y `terraform plan` sobre el HCL generado.
- **Arquitectura modular y extensible:** se materializa en la pipeline de extracción (Sección ??) y en el modelo de dominio (Sección ??), que aíslan el catálogo del proveedor del núcleo de la aplicación y permiten incorporar nuevos proveedores sin modificarlo.

Respecto al estado del arte, la propuesta se sitúa en el hueco identificado en la Tabla ??: combina modelado visual, independencia de proveedor, licenciamiento open source, generación de IaC editable y ejecución local. Las soluciones existentes cubren subconjuntos parciales de esos criterios, pero ninguna los reúne simultáneamente, lo que confirma la pertinencia de la propuesta.

15.3. Limitaciones y trabajo futuro

Aun cumpliendo los objetivos planteados, el alcance del MVP deja limitaciones explícitas y abre varias líneas naturales de evolución.

Limitaciones actuales

- **Distribución limitada a Linux:** el desarrollo y la validación se han realizado sobre Linux. Aunque Tauri soporta macOS y Windows de forma nativa, el empaquetado, la firma y la distribución estable en esas plataformas requieren trámites (certificados, instaladores, actualizaciones) ajenos al núcleo técnico del proyecto.
- **Análisis estático de seguridad pendiente:** la planificación inicial contemplaba complementar la validación funcional con un análisis estático del HCL generado mediante herramientas como Trivy, orientado a detectar configuraciones inseguras (por ejemplo, grupos de seguridad demasiado abiertos o buckets públicos por defecto). Esa integración no se ha podido completar dentro del marco temporal del proyecto y queda como tarea pendiente, no como línea de evolución del producto.
- **Soporte multi-proveedor.** La pipeline está diseñada para ejecutarse sobre cualquier proveedor de Terraform; queda pendiente generar los catálogos de Google Cloud y Microsoft Azure y adaptar las reglas de mapeo específicas de cada uno.
- **Flujos avanzados de Terraform fuera del MVP:** *workspaces*, *backends* remotos y mecanismos similares no están integrados en la experiencia visual. Son funcionalidades operativas más cercanas a entornos de producción que al objetivo de aprendizaje y prototipado del MVP.

Líneas de trabajo futuro

- **Integración con un servidor MCP (*Model Context Protocol*) [36].** Exponer la aplicación como servidor MCP permitiría que asistentes de IA pudieran inspeccionar el modelo de dominio, proponer cambios sobre la arquitectura visual y consumir el catálogo de recursos directamente, abriendo un flujo de trabajo asistido sin sacrificar el control del usuario.

- **Sincronización del proyecto con un repositorio Git.** El formato de proyecto ya es legible y diff-able; falta integración nativa para vincular cada vista con un repositorio, registrar commits automáticos en puntos significativos del ciclo de vida y permitir colaboración multi-usuario con resolución asistida de conflictos.
- **Soporte para módulos Terraform.** Permitiría encapsular sub-arquitecturas reutilizables como nodos compuestos en el lienzo, aumentando la productividad en proyectos grandes.
- **Importación de infraestructura existente.** Un asistente para `terraform import` permitiría reconstruir el modelo a partir de una infraestructura ya desplegada, ampliando el público objetivo a equipos con infraestructura preexistente.
- **Ejecución automatizada de la pipeline de extracción.** Actualmente la pipeline se ejecuta de forma manual ante cada cambio de versión del proveedor y los catálogos resultantes se importan al repositorio de la aplicación. Como evolución natural, se plantea integrarla en un flujo de GitHub Actions [37] que detecte nuevas versiones de los proveedores de Terraform (o se dispare de forma periódica), ejecute la pipeline completa de forma reproducible y abra un *pull request* con los catálogos y plantillas actualizadas. Esto elimina por completo el paso manual de importación y mantiene el catálogo distribuido sincronizado con la última versión estable de cada proveedor.

15.4. Valoración personal

En lo personal, el balance del proyecto es muy positivo. Una de las motivaciones iniciales era **profundizar en Terraform y en el ecosistema Cloud** desde un ángulo distinto, y construir esta herramienta me ha obligado a entender Terraform a un nivel que difícilmente habría alcanzado limitándome a usarlo: leer y procesar su schema, integrar su CLI como subprocesso, gestionar sus flujos interactivos y reconstruir el sentido de su sintaxis para traducirla desde un modelo visual.

En paralelo, el proyecto me ha permitido **aprender y consolidar Rust** como lenguaje de sistemas para el backend nativo, y **ganar soltura en el desarrollo frontend** moderno con React y TypeScript. Llegar a tener las dos partes comunicándose de forma fluida a través de Tauri, con una capa de infraestructura sólida en Rust, ha sido uno de los aprendizajes más valiosos del TFG.

Más allá del aprendizaje técnico, una motivación importante ha sido **aportar algo a la comunidad open source del Cloud**. Las herramientas visuales actuales son mayoritariamente propietarias o están ligadas a un proveedor; poder contribuir con una alternativa abierta, neutral y orientada al aprendizaje, aunque sea como punto de partida, es algo que me satisface en lo profesional y en lo personal.

Referencias

- [1] *¿Qué es el cloud computing? / IBM*. 14 de feb. de 2024. URL: <https://www.ibm.com/es-es/think/topics/cloud-computing> (visitado 23-02-2026).
- [2] *¿Qué es la infraestructura como código (IaC)? ¿Cómo funciona el IAC?* Fortinet. URL: <https://www.fortinet.com/lat/resources/cyberglossary/infrastructure-as-code.html> (visitado 23-02-2026).
- [3] *¿Qué es AWS?* Amazon Web Services, Inc. URL: <https://aws.amazon.com/es/what-is-aws/> (visitado 23-02-2026).
- [4] *Descripción general de Google Cloud / Get started*. Google Cloud Documentation. URL: <https://docs.cloud.google.com/docs/overview?hl=es-419> (visitado 23-02-2026).
- [5] *¿Qué es Azure? / Microsoft Azure*. URL: <https://azure.microsoft.com/es-es/resources/cloud-computing-dictionary/what-is-azure/> (visitado 23-02-2026).
- [6] *IBM Cloud*. URL: <https://www.ibm.com/es-es/solutions/cloud> (visitado 25-02-2026).
- [7] *Alibaba Cloud-Alibaba Group*. URL: <https://home.alibabagroup.com/en-US/about-alibaba-businesses-1747835448811585536> (visitado 25-02-2026).
- [8] *Hyperscalers - hiperescaladores: principales proveedores de la nube*. URL: <https://www.redhat.com/es/topics/cloud-computing/what-is-a-hyperscaler> (visitado 23-02-2026).
- [9] *Escalabilidad - Definicion.de*. Definición.de. URL: <https://definicion.de/escalabilidad/> (visitado 23-02-2026).
- [10] *Portabilidad*. Cloud Native Glosario. URL: <https://glossary.cncf.io/es/portability/> (visitado 23-02-2026).
- [11] *¿Qué es la sostenibilidad? Definición e importancia / Sostenibilidad de Microsoft*. URL: <https://www.microsoft.com/es-co/sustainability/learning-center/sustainability-definition> (visitado 24-02-2026).
- [12] *¿Qué es el open source o código abierto?* URL: <https://www.redhat.com/es/topics/open-source/what-is-open-source> (visitado 24-02-2026).
- [13] *¿Qué es Terraform? / IBM*. 15 de abr. de 2025. URL: <https://www.ibm.com/es-es/think/topics/terraform> (visitado 26-02-2026).
- [14] *Brainboard - Aplicación de escritorio para Mac, Windows (PC)*. WebCatalog. URL: <https://webcatalog.io/es/apps/brainboard> (visitado 26-02-2026).
- [15] *Infrastructure as Code*. pulumi. URL: <https://www.pulumi.com/docs/iac/> (visitado 05-05-2026).
- [16] *Cloud Development Framework - AWS Cloud Development Kit - AWS*. Amazon Web Services, Inc. URL: <https://aws.amazon.com/cdk/> (visitado 19-05-2026).
- [17] *¿Qué es el MVP o producto mínimo viable y para qué sirve?* Banco Santander Impulsa Empresas. URL: <https://www.impulsa-empresa.es/diccionario/producto-minimo-viable/> (visitado 26-02-2026).

- [18] Henrik Kniberg y Mattias Skarin. *Kanban and Scrum - Making the Most of Both*. Google Books-ID: Hx1KAgAAQBAJ. Lulu.com, 2010. 122 págs. ISBN: 978-0-557-13832-6.
- [19] *Acerca de GitHub y Git - Documentación de GitHub*. GitHub Docs. URL: <https://docs-internal.github.com/es/get-started/start-your-journey/about-github-and-git> (visitado 26-02-2026).
- [20] *Qué es Trello, características y ventajas | OpenWebinars*. OpenWebinars.net. URL: <https://openwebinars.net/blog/que-es-trello/> (visitado 26-02-2026).
- [21] *¿Qué es un diagrama de Gantt y para qué sirve? | OBS Business School*. URL: <https://www.obsbusiness.school/blog/que-es-un-diagrama-de-gantt-y-para-que-sirve> (visitado 17-03-2026).
- [22] *Visual Studio Code - The open source AI code editor*. URL: <https://code.visualstudio.com/> (visitado 17-03-2026).
- [23] *Rust Programming Language*. URL: <https://rust-lang.org/> (visitado 17-03-2026).
- [24] *¿Qué es Tauri?* Tauri. 19 de feb. de 2025. URL: <https://v2.tauri.app/es/start/> (visitado 17-03-2026).
- [25] *Getting Started*. vitejs. URL: <https://vite.dev> (visitado 17-03-2026).
- [26] *React*. URL: <https://react.dev/> (visitado 19-05-2026).
- [27] *JavaScript With Syntax For Types*. URL: <https://www.typescriptlang.org/> (visitado 19-05-2026).
- [28] *LaTeX - A document preparation system*. URL: <https://www.latex-project.org/> (visitado 17-03-2026).
- [29] *Google Meet*. URL: <https://meet.google.com/landing> (visitado 17-03-2026).
- [30] *hashicorp/hcl*. original-date: 2014-07-31T21:09:00Z. 17 de mar. de 2026. URL: <https://github.com/hashicorp/hcl> (visitado 17-03-2026).
- [31] *Calcular sueldo medio por profesión y provincia | InfoJobs Salarios*. URL: <https://salarios.infojobs.net/> (visitado 09-03-2026).
- [32] *Precio de la tarifa de luz por horas HOY | 09 Marzo 2026*. URL: <https://tarifaluzhora.es/> (visitado 09-03-2026).
- [33] *Home*. C4 model. URL: <https://c4model.com/> (visitado 19-05-2026).
- [34] *Build cross-platform desktop apps with JavaScript, HTML, and CSS | Electron*. URL: <https://electronjs.org/> (visitado 19-05-2026).
- [35] *The Go Programming Language*. URL: <https://go.dev/> (visitado 05-05-2026).
- [36] *What is the Model Context Protocol (MCP)?* Model Context Protocol. URL: <https://modelcontextprotocol.io/docs/getting-started/intro> (visitado 19-05-2026).
- [37] *Descripción de GitHub Actions - Documentación de GitHub*. GitHub Docs. URL: <https://docs-internal.github.com/es/actions/get-started/understand-github-actions> (visitado 19-05-2026).